COMSATS University Islamabad

Abbottabad, Pakistan

Study Buddy:

A unified learning platform

By

Waseem Riaz	**CIIT/FA22-BSE-055/ATD**
Hamza Gul	**CIIT/FA22-BSE-086/ATD**
Shahzaib Khan	**CIIT/FA22-BSE-100/ATD**

Supervisor

Sumair Khan

Bachelor of Science in Software Engineering (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

COMSATS University, Islamabad Pakistan

Study Buddy:

A unified learning platform

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Software Engineering (2022-2026)

By

Waseem Riaz	**CIIT/FA22-BSE-055/ATD**
Hamza Gul	**CIIT/FA22-BSE-086/ATD**
Shahzaib Khan	**CIIT/FA22-BSE-100/ATD**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Waseem Riaz	**Hamza Gul**	**Shahzaib Khan**
----------	----------	----------

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) "Study Buddy: A unified learning platform" was developed by **Waseem Riaz (CIIT/FA22-BSE-055)**, **Hamza Gul (CIIT/FA22-BSE-086)** and **Shahzaib Khan (CIIT/FA22-BSE-100)** under the supervision of "**Sumair Khan"** and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor

External Examiner

Head of Department

(Department of Computer Science)

DEPARTMENT OF COMPUTER SCIENCE,

COMSATS UNIVERSITY ISLAMABAD

ABBOTTABAD CAMPUS

FINAL APPROVAL

This is to certify that we have read the thesis submitted by **Waseem Riaz** having registration number **CIIT/FA22-BSE-055/ATD** in the session **SP26**. It is our judgment that this thesis is of sufficient standard to warrant its acceptance by the COMSATS University Islamabad, Abbottabad Campus for the **Bachelor of Software Engineering.**

1. External Examiner Umar

Umar Aftab Qureshi

Executive Manager

EasyPaisa Digital

2. Internal Examiner _____

Dr. Syed Faraz Ahmed

3. Supervisor _____

Sumair Khan.

4. Head of Department _____

Dr. Zia ur Rehman

Department of Computer Science,

COMSATS UNIVERSITY ISLAMABAD,

Abbottabad Campus.

Dated: 11/08/26

DEPARTMENT OF COMPUTER SCIENCE,

COMSATS UNIVERSITY ISLAMABAD

ABBOTTABAD CAMPUS

FINAL APPROVAL

This is to certify that we have read the thesis submitted by **Hamza Gul** having registration number **CIIT/FA22-BSE-086/ATD** in the session **SP26**. It is our judgment that this thesis is of sufficient standard to warrant its acceptance by the COMSATS University Islamabad, Abbottabad Campus for the **Bachelor of Software Engineering.**

1. External Examiner _____

Umar Aftab Qureshi

Executive Manager

EasyPaisa Digital

2. Internal Examiner _____

Dr. Syed Faraz Ahmed

3. Supervisor _____

Sumair Khan.

4. Head of Department _____

Dr. Zia ur Rehman

Department of Computer Science,

COMSATS UNIVERSITY ISLAMABAD,

Abbottabad Campus.

Dated: 11/08/26

DEPARTMENT OF COMPUTER SCIENCE,

COMSATS UNIVERSITY ISLAMABAD

ABBOTTABAD CAMPUS

FINAL APPROVAL

This is to certify that we have read the thesis submitted by **Shahzaib Khan** having registration number **CIIT/FA22-BSE-100 /ATD** in the session **SP26**. It is our judgment that this thesis is of sufficient standard to warrant its acceptance by the COMSATS University Islamabad, Abbottabad Campus for the **Bachelor of Software Engineering.**

1. External Examiner _____

Umar Aftab Qureshi

Executive Manager

EasyPaisa Digital

2. Internal Examiner _____

Dr. Syed Faraz Ahmed

3. Supervisor _____

Sumair Khan.

4. Head of Department _____

Dr. Zia ur Rehman

Department of Computer Science,

COMSATS UNIVERSITY ISLAMABAD,

Abbottabad Campus.

Dated: 11/08/26

EXECUTIVE SUMMARY

Academic tutoring solutions worldwide are plagued with one big problem: there's a lack of a single place to source, teach, and meet tutors that offer both structured mentorship and peer-to-peer collaboration. Current tools meet these needs individually, and require the user to develop and maintain switching mental maps. Current tools deal with each need on its own, requiring learners to maintain multiple mental maps.

StudyBuddy addresses this issue by tightly knitting together every facet of the academic experience into a unified, web-based environment for students and instructors. StudyBuddy's essence lies in generating insights from content using Artificial Intelligence, which automates and demystifies the preparation process. Students are able to input their study information and this is then converted into structured notes, short information summaries and questions in no time at all. This saves time on manual input and enables students to spend more time on learning of important concept.

The other pillar is collaboration. A rule based matching system matches students who have similar academic interests and allows them to meet others in real time and use video/audio technology via WebRTC when they need help. Users can exchange documents, monitor progress, and monitor over time and continue to collaborate.

Those wanting to be taught by a professional can learn about the instructors, set up a meeting time and communicate with them in a structured System through the mentorship system.

StudyBuddy also features a Focus Room, which utilizes timers based on the Pomodoro technique, to help maximize productivity by striking a balance between study and break times. Progress in these sessions is monitored and analysed, to inform users about their learning patterns and to enhance them over time.

It's built with a modern tech stack, where the front-end is React, and the back-end is Nodejs based with Express.It uses a modern tech stack and features a React front end, a Nodejs and Express back end, and MongoDB for data storage. Real-time communication is achieved through WebRTC technology, and AI tasks processed locally using an Ollama-hosted language model, eliminating the need for external APIs.

In addition to user interactions, StudyBuddy encourages community practice through discussion forums and a Resource Hub in which sharing and finding study materials are possible. An admin panel supports the integrity of the platform, such as managing users; verifying instructors, and anything related to system activity.

In summary, StudyBuddy fills the gaps in existing education resources by providing an integrated platform that integrates AI-based support, peer learning, coaching, and peer interaction.

ACKNOWLEDGEMENT

Praise is due to ALLAH SWT, who has blessed us with a small part of His vast knowledge and given us the strength and ability to complete this difficult task.

We are sincerely thankful to our project supervisor, Mr. **Sumair Khan**, for his guidance, valuable advice, and continuous support throughout this project. His supervision and encouragement helped us at every stage of our work and played an important role in the successful completion of this project. We are truly grateful for his time, patience, and support.

We would also like to thank our parents and families, who were always there to support and encourage us throughout this journey. Their constant support and guidance helped us stay motivated and taught us the importance of honesty and hard work.

Waseem Riaz **Hamza Gul** **Shahzaib Khan**

---------- ---------- ----------

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BR	Business Rule
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
EdTech	Educational Technology
FR	Functional Requirement
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token
LLM	Large Language Model
MVP	Minimum Viable Product
NoSQL	Non-Relational Structured Query Language
OAuth	Open Authorization (2.0)
OE	Operating Environment
PDL	Program Design Language
RTC	Real-Time Communication
SDK	Software Development Kit
SDD	Software Design Description
SRS	Software Requirements Specification
SSD	System Sequence Diagram
UC	Use Case
UML	Unified Modeling Language
WebRTC	Web Real-Time Communication
XP	Experience Points

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. Introduction

This chapter lays the foundation of the motivation and structure of the StudyBuddy project. It sets out the basic scope, discusses the academic background, describes the problem background and specifies the software development lifecycle selected.

1.1. Brief

StudyBuddy is a web-based peer learning and mentorship ecosystem that was designed to integrate the various academic support tools that exist into a single platform. It is aimed at addressing the lack of accessible, affordable, and systematic educational support, especially for students in less-resourced areas who use disjointed, unorganized resources to support their learning, note-taking, and peer engagement.

In essence, StudyBuddy has four unique features: AI-generated content, collaborative in real time, personalised mentorship in a structured format, and community knowledge sharing.In short, StudyBuddy combines four key features: AI-generated content, real-time peer collaboration, structured one-on-one mentorship, and community knowledge sharing. Students can use a cloud-based gemma 3 intelligent content engine to feed topics and get structured notes, condensed summaries and auto-generated quizzes as they go. A rule-based peer matching engine is used to find online students that have overlapping areas of interest in their subjects and connect them through live video and audio sessions using WebRTC. The mentorship subsystem enables students to explore the verified profiles of instructors, view the availability of mentors and request a mentoring session which the instructor then manages via an exclusive control panel.

Using a React frontend, you will provide the client-side experience, and a Node.js and Express backend will handle all business logic and requests. The main data storage is the MongoDB used to store the user's profile, their session data, community posts, content & resource generated metadata. Platform integrity is top-to-bottom managed by an administrative layer which allows staff to validate instructors, control user accounts, work through reported incidents and view platform-wide analytics and reports from a single dashboard.

1.2. Relevance to Course Modules

This project integrates core knowledge and principles of different academic modules (covered during Software Engineering Bachelor of Science degree) to create a working and scalable maintainable Web Application.

1.2.1. Web Development

StudyBuddy is designed using the full-stack approach of React and Node.js: React will be used to build dynamic user-facing components like the dashboard, study rooms, and community feed, while Node (with Express) will handle routing, processing middleware, and dispatching API output from one responsible back-end service.

1.2.2. Object Oriented Programming

The system is clearly divided into modules using the OOP approach. Each of these independently enabled features is implemented in a separate platform, with its own controller, service layer, data model, and encapsulated in the codebase according to the single responsibility principle, and accessible through code via the separate access points (e.g., forums, resource management etc.).

1.2.3. Computer Networks

WebRTC provides study buddy peer collaboration and mentorship sessions with low latency, browser native video and audio communication. All WebSocket signalling to connect different peers, handling session tokens, and processing change of participants' state are done in real time in the back end – without the need for third party media servers during initial implementation.

1.2.4. Human Computer Interaction

The User interface for each module is designed to be clean and free from distractions with the help of the Tailwind CSS. The Focus Room features a minimalist design that directs attention toward the activity to be performed with the least cognitive load, while the dashboard presentation delivers personalised progress information, reminders of the next session and intuitive controls, offering smooth user experience and low learning curve for new users.

1.2.5. Database Systems

StudyBuddy relies on the NoSQL backend, MongoDB, which can hold many different kinds of data structures including AI-generated content arrays, nested session metadata, and community post threads. mongoose schemas are defined at the application level, and used to validate and shape the data in all collections

1.2.6. Software Engineering Concepts

The project has been developed within an Agile iterative approach and goes through the phases of the platform's Minimum Viable Product (MVP) sprints. Each sprint brings a testable and verifiable increment of requirements gathering, system design, implemented system and system verification/validation including system integration testing that begin with a question raised in the use case specification and end with system integration testing.

1.2.7. Software Design and Architecture

There are 3 layers to the Software Design and Architecture StudyBuddy - it uses React for the UI layer, Node.js and Express for the application logic layer and MongoDB for the data layer.This separation means that each tier can be operated, tested, and scaled separately.

The back end is developed using a controller-service-model design pattern, with controllers responsible for validating incoming requests and routing them to the appropriate service, services that represent business logic and models representing database schemas, making it easy to read and extend for new features.

1.3. Project Background

Students today have no shortage of educational technology, they just need one place where the technology all hangs together. A typical student is using a tutoring app on their cell phone, a note taking app as well, and a group chat to coordinate study sessions. All of the current platforms address one aspect of the problem individually. AI note generators don't have any peer collaboration layer. Discussion forums do not provide any real-time sessions or mentorship. There is no connection between these tools and as time goes on, the student will lose his concentration and will lose time dedicated to learning.

The disconnect is most stark when it's the worst time of day; just before a test, when a student needs help now and has no practical way to get it from an AI helper, a peer, and someone who is already an instructor.

StudyBuddy was developed with that in mind. A student logs in and all these various features of AI content generation, peer matching, mentor booking panel, a community forum, a resource hub and a distraction-free Focus Room are all brought together. Behind the scenes, there's an administrative section that allows the staff to verify instructors, moderate accounts, monitor the activity on the platform and more… ensuring the platform remains trustworthy as it grows.

1.4. Literature Review

Market solutions are typically in two categories today. Fitness trackers are best at tracking exercise-related data such as calories burned and heart rate. They don't have the facilities for a vehicle commute or to estimate fuel. They offer great navigation tools and traffic information. They don't provide ongoing, detailed personal logs or social community elements. Table 1.1 shows the market leaders segmented and their limitations compared to our proposed solution.

Table 1.1: Literature Review/System Analysis

Application Name	Limitations of Existing Systems	StudyBuddy Solution
Khan Academy	Focuses only on self-paced video learning. Lacks mentorship, peer interaction, or AI powered content creation.	StudyBuddy adds structured mentorship, AI note/quiz generation, and peer matching features for collaborative study.
Noon Academy	Focused on live classes and group quizzes, but limited AI features. No built-in marketplace or structured mentorship.	StudyBuddy enhances the learning model by introducing personalized one-on-one mentorship, an interactive marketplace for educational resources.
Maqsad	Content-heavy (videos/quizzes) but lacks peer interaction and mentorship. No community driven learning.	StudyBuddy adds real time peer matching, community forums, and structured mentor support alongside AI note generation.

1.5. Analysis from Literature Review

While the market's platforms are many, a closer look on the most popular ones reveals an unfortunate pattern which is one problem solved, everything else left out: students left to solve the rest.

Brainly takes the peer-to-peer Q&A space and does an okay job of reaching students who ask questions with those who may have answers. But the one big problem is, the platform doesn't do any quality control on those answers. If a student gets a response that is factually wrong but confidently written, they will not be able to tell this is an error in Brainly. No structured mentorship, no confirmed educator involvement, and no learning pathway beyond the one on one question/answer exchange. StudyBuddy does it differently by using AI-generated content, expert teacher guidance and study groups to provide a high-quality academic support system that isn't solely dependent on the first teacher who saw the question.

The polar opposite to Khan Academy is a very well-crafted and high-quality piece of content, presented in a strictly one-way format. Students view, pause and re-watch video lessons, but the platform does not provide a way to post a follow-up question.

No one to talk to while they are working through a problem, no one to ask questions of while the video explanation doesn't land, and no teacher to reference when the video explanation doesn't land. The biggest drawback to this platform is the idea that all

learners can and should watch video content at their own rate and that this is all they need. StudyBuddy does not require a passive model; it is instead an active option. When the first two don't help, the mentorship module gives them access to a verified instructor on the same topic as them.When these two approaches do not solve the difficulty, the personalised notes and quizzes generated by the AI Content Generator connect them to someone who is working on the same topic at the same time.

Quizlet shifts the burden of preparation onto the student, before any studying can come about. The only way for a student to make use of the platform is to create their own sets of flashcards from scratch, just taking up the time and effort that should be dedicated to actual revision. In addition, Quizlet does not provide any community feature, live collaboration, mentoring or sharing/monetising of good study content. StudyBuddy eliminates all of the preparation burden. The Content Generator will take a topic or raw study material, and provide structured notes, summaries, and study questions automatically. The built-in resource marketplace and community forum enhances the experience, going far beyond any flashcard tool.

Noon Academy motivates students through engaging, live lessons from the teacher and through quizzes in groups. The drawbacks are that students are limited in how they can take ownership of the learning process (instructors set up sessions, minimize use of AI, no 1-on-1 mentoring, and students can't self-organize peer sessions outside of the class format). With StudyBuddy, students have both of these opportunities at the same time. Mentor led sessions are offered for booking and can take place at times convenient for both parties; peer matching is offered to allow students to organise their own live study sessions independently. In addition, a resource marketplace is built beside both, enabling students and instructors to share and access study materials without having to wait until a class is scheduled to distribute them.

Maqsad has a contextual edge over generic platforms as it provides video and quiz content that is localised, relevant and specific to the Pakistani context for the students. The experience is however totally alone. The lack of peer identification for students learning the same material, community forum for discussion, real-time collaboration, and structured mentorship is beyond the pre-recorded content itself. StudyBuddy tackles each of these head-on: The peer matching engine matches students who have similar interests in the subjects they study, allowing for live sessions to connect students, the community forum allows students to discuss anything that occurs asynchronously, and verified instructors are available to provide one-on-one guidance that a pre-recorded video cannot replace.

This review lost repetitions seem to make clear that all of the platforms already in place are in a thin range. None of them attempt to provide a seamless experience of integrating AI-generated content, live collaboration with peers, organized mentorship, community interactions, and a shared resource hub. StudyBuddy was created with the belief that students shouldn't have to work out their own support system using five separate tools: it should all be in one place.

1.6. Methodology and Software Lifecycle

StudyBuddy has been designed using a hybrid approach; a blend of the Structured Procedural Design methodology and Agile Scrum framework. The approach chosen: The structuring approach was carefully chosen; rather than the typical object oriented / class oriented structuring, the approach chosen for the platform's logic is well understood functions and data flows, where each function receives the input, performs a series of actions and returns the output of the function. This breakdown was a natural fit along with the design of the system in the SDD such that each feature was described as a series of step by step algorithms expressed in PDL (procedural decomposition). The agile approach enabled firms to produce and test each module one at a time to assure that the nine key features of the new system were realized step-by-step, rather than all in one shot.

1.6.1. Flexibility

Academic platforms grow during development, as the users' needs become increasingly apparent. StudyBuddy's needs evolved over a number of iterations – from a cloud-based DeepSeek API to a locally hosted Ollama model and, during development, the introduction of an administrative module that was essential to the original scope. These transitions were made manageable in using a structured approach as each feature was implemented as a stand-alone set of functions with defined input and output. Replacing or extending one function doesn't lead to unanticipated parts of system cascading. Within the scope changes, agile was able to incorporate new requirements into subsequent sprints, with no disruption of modules that were completed and tested during previous sprints.

1.6.2. Early Feedback

At the end of each sprint, workable software was created instead of testing anything significant until the last few sprints. Once each increment was ready for use, it was checked against the original use case specifications. This continuous evaluation process uncovered alignment problems early, such as the mentor booking process and the peer session initiation sequence: the first time this was implemented it behaved in a way that conflicts with the intended behaviour documented in the SRS use case descriptions.

1.6.3. Rapid Prototyping

Early prototyping of the same complex features helped a lot in their full implementation. The WebRTC-based peer session system involved understanding the full signalling path — ICE candidate exchange, generation of session tokens, handling connections — prior to any production code was written. This is accomplished very nicely with the structured approach, and, indeed, the points at which the process fails can be identified, prior to implementing the process, by visualizing the entire process as a series of named function calls in PDL.

1.6.4. Continuous Improvement

Functionality and code review time were conducted at the end of each sprint. Throughout the testing, performance of the database queries was regularly rechecked as collections were growing in the database, and the procedural service functions were refactored throughout several sprints to remove redundant operations and speed up response time. The AI content generation pipeline was also improved incrementally, with prompt structures tweaked, response parsing enhanced, and handling errors when models are unavailable strengthened — all of this done in a series of small, iterative improvements instead of in one large shot at the end.

1.6.5. Adaptability

There are constraints that are not necessarily considered in other development areas that must be taken into account during the web application development. Initial testing showed that real-time WebRTC sessions were more sensitive to variability in the network than initially thought, also the peer matching function was found to be too broad with just subject based filtering as soon as the test was launched. Each of these problems was isolated to a specific named function, and so fixes could be applied to the function without having to worry about side effects elsewhere in the system as they were involved in an interconnected class hierarchy.

1.6.6. Collaborative Approach

The three-member team split its work between the nine modules of the platform, with each member owning a set of clearly defined procedural functions, but each member being responsible for integration compatibility to the others. Each function had clear inputs and outputs and there is no hidden state within a function that is modified by that function that may affect others in the same program; the design was structured so that one person could pass a populated function to another and simply select an interface in a single moment without needing to worry about a changeable state within that function. Both the frontend's API calls, and the actual backend service functions were always kept in sync by periodic "coordinating" them, since both had a development cycle during the sprints.

1.6.7. Incremental Delivery

The order in which the development sequence occurred was carefully sequence to ensure that there was a solid foundation on which to add more and more complicated functions. The user authentication and the core dashboard were finished first, and then the core dashboard was built on a reliable foundation, which could be expanded upon by each following module. The Content Generator and peer matching system came first, followed by the mentorship booking flow and then the community forum and resource hub, and lastly the administrative layer. This methodology was applied, thereby reducing the number of failures in the last integration phases as new modules were integrated in a functional, stable system.

2. Problem Definition

This chapter contains a brief description of the specific problems being solved by StudyBuddy and defines the outcomes of the project. It brings to the fore issues of the fragmentation of available educational support solutions and catalogs the distinct products and solutions that must be developed to meet the needs of a single solution for peer learning and mentorship.

2.1. Problem Statement

The issue cited above, where academic support cannot be obtained during self-study in a timely and cost-effective manner and easily is a problem facing students all across the world, especially in those places where academic support is not available. Limited mentorship, fragmented resources, too costly tutoring and poor self-motivation all pose a challenge for learners to be consistent and make good academic gains. There are different sites where you can get tutoring service, write something or have an academic discussion, but none of them of these sites can be able to give you an integrated and holistic learning space. Likewise, traditional platforms in online learning do not grasp the importance of the ‘engagement systems’ that allow students to have consistency and accountability in their study routine and stay motivated at all times in their online learning path.

2.2. Deliverables and Development Requirements

The product outputs information assets and development environment that must be precisely defined to ensure the product's success. The following concrete outputs of the Studybuddy project and the technologies behind them are presented.

2.2.1. Deliverables

StudyBuddy will be a web application usable with any modern web browser, as opposed to having to be installed. This is the main output that will be a responsive web app, and will be the primary focus for students and teachers to access all other features on the platform, including AI Content Generator, Focus Room, Study Rooms, peer matching, mentorship booking, community forum and the Resource Hub.

The main app also includes a dedicated admin interface, which lets system administrators keep track of platform analytics, approve or block mentors, manage teacher requests, and go through the reported issues.

All these applications are linked to a secure backend system. It also embraces a back end with NodeJS and ExpressJS for authentication, session keeping, role-based authorization, and communication signaling for real-time communications.

We are using Mongodb to store all the structured data as lists of users, mentorship sessions, etc etc which are generated by the backend.

2.2.2. Development Requirements

There are software environments and frameworks that are required to build a collaborative, AI-integrated, real-time learning platform. Requirements are broken down into conceptual tools and underlying technologies to preserve the architectural clarity.

Table 2.1: Development Tools Requirements

Tool	Version	Rationale
Visual Studio Code	1.104	Provides a comprehensive code editor for frontend (React/Flutter) and backend (Spring Boot) development.
Postman	11.54.6	For testing of REST APIs and verification of backend services.
Swagger	3.1.0	Generates interactive API documentation & direct API testing.
MS Office	2021	Supports the creation of formal project documentation and presentation materials.
Figma	2026	Allows for the creation of UI/UX designs.
Git	2.51.0	Manages source code version control locally and collaboratively.

Table 2.2: Development Technologies Requirements

Technologies	Version	Rationale
React	18.3	Powers the web frontend, enabling the construction of scalable, component-based, and responsive user interfaces.
Node.js	20.11	Serves as the core runtime environment for the backend, supporting asynchronous and event-driven server operations.
Express.js	4.18	Provides a lightweight and flexible web framework for building RESTful API routes and managing backend business logic.
MongoDB	7.0	Acts as the primary NoSQL database, storing user profiles, session records, community posts, and AI-generated content metadata.
Tailwind CSS	3.4	Enables rapid and consistent UI styling through a utility-first CSS framework aligned with the StudyBuddy design system.
WebRTC / Agora SDK	4.20	Facilitates seamless, low-latency real-time video and audio communication for study rooms and peer sessions.
Ollama (Gemma3)	0.3	Hosts the locally deployed large language model responsible for AI-based note generation, summarization, and quiz creation.
JWT	9.0	Enforces secure, token-based authentication and role-based authorization across all protected API routes.

2.3. Current System

Academic learning, in today's context, requires students to deal with disorganized toolsets that do not talk to each other. To study for a test, a student must also deal with multiple disconnected platforms, all of which have a limited function with little or no knowledge of the others. Note taking is in one app, quiz practice in another, mentoring in a different channel and collaboration among peers using another application completely. This division of study time can be detrimental to students who are trying to establish a consistent academic routine, as well as waste valuable study time.

One part of this market is the use of AI tools in education.AI tools for education are one part of this market. There have been a number of platforms that have spent a lot of money building content and automated practice. Platforms like Quizlet and Khan Academy have made a huge investment in content and automated practice. They are based on an underlying design, however, which assumes a single learner consuming pre-packaged, rather than creating their own learning. The systems have limited opportunity for real-time peer collaboration, structured mentorship and community-based knowledge transfer. It is impossible for a student to discuss the results with a classmate or book a session with a subject expert in the same context after completing a practice quiz.

A contrasting disadvantage is that mentorship and tutoring platforms are limited. This type of services includes links to qualified teachers, and serves mainly as a scheduling and video conferencing wrapper. They have no built-in study tools, no AI-generated content, and no way for students to locate suitable study partners out of a formal paid study session. When a session is over, the learner goes back to the same disjointed ecosystem from which they came.

A third, also very separate, are community forums and academic discussion boards. These are designed for purposes of information sharing and peer-to-peer support, but don't provide the productivity tools serious learners require. A student can ask a question and receive the reply there; a student cannot set an agenda of focused study, take some notes and then receive an in-depth response from a mentor to the question from the same location.

To address this head-on is what StudyBuddy is all about, by achieving this through consolidating peer-work, AI-generated resources, structured mentorship, involvement in the community, and productivity tools within one platform. The system enables the seamless flow from creating notes to mentoring bookings to community discussions to study rooms, and a motivating and cohesive learning experience all along the way.

Table 2.3: Comparison of Existing Systems

System Category	Core Capabilities	Major Limitations
Q&A Learning Platforms (e.g., Brainly)	Quick question-answer support, large user base, fast responses.	Inconsistent answer quality, no structured mentorship, lacks personalized learning paths.
Self-Paced Learning Platforms (e.g., Khan Academy)	High-quality video lectures, structured courses, self-paced progression.	Limited peer interaction, no mentorship system, lacks AI-driven content generation
Flashcard-Based Tools (e.g., Quizlet)	Effective memorization through flashcards, user-generated content, simple interface.	No strong community engagement, lacks mentorship, limited beyond memorization techniques.
Live Learning Platforms (e.g., Noon Academy)	Real-time classes, interactive quizzes, group-based learning sessions.	Limited AI integration, no resource marketplace, weak structured mentorship support.
Content-Based Platforms (e.g., Maqsad)	Rich educational content (videos, quizzes), curriculum-focused learning.	Minimal peer collaboration, no community-driven interaction, lacks mentorship features

3. Requirement Analysis

This chapter represents the functional and non-functional requirements of the StudyBuddy web application in a formal way. System behaviors and performance limitations are mapped out and a concrete system blueprint is developed.

3.1. Use Cases Diagram

The figure 3.1 Use Case Diagram gives a complete view of the functionality of the website and depicts relations and interactions between the users and the system.

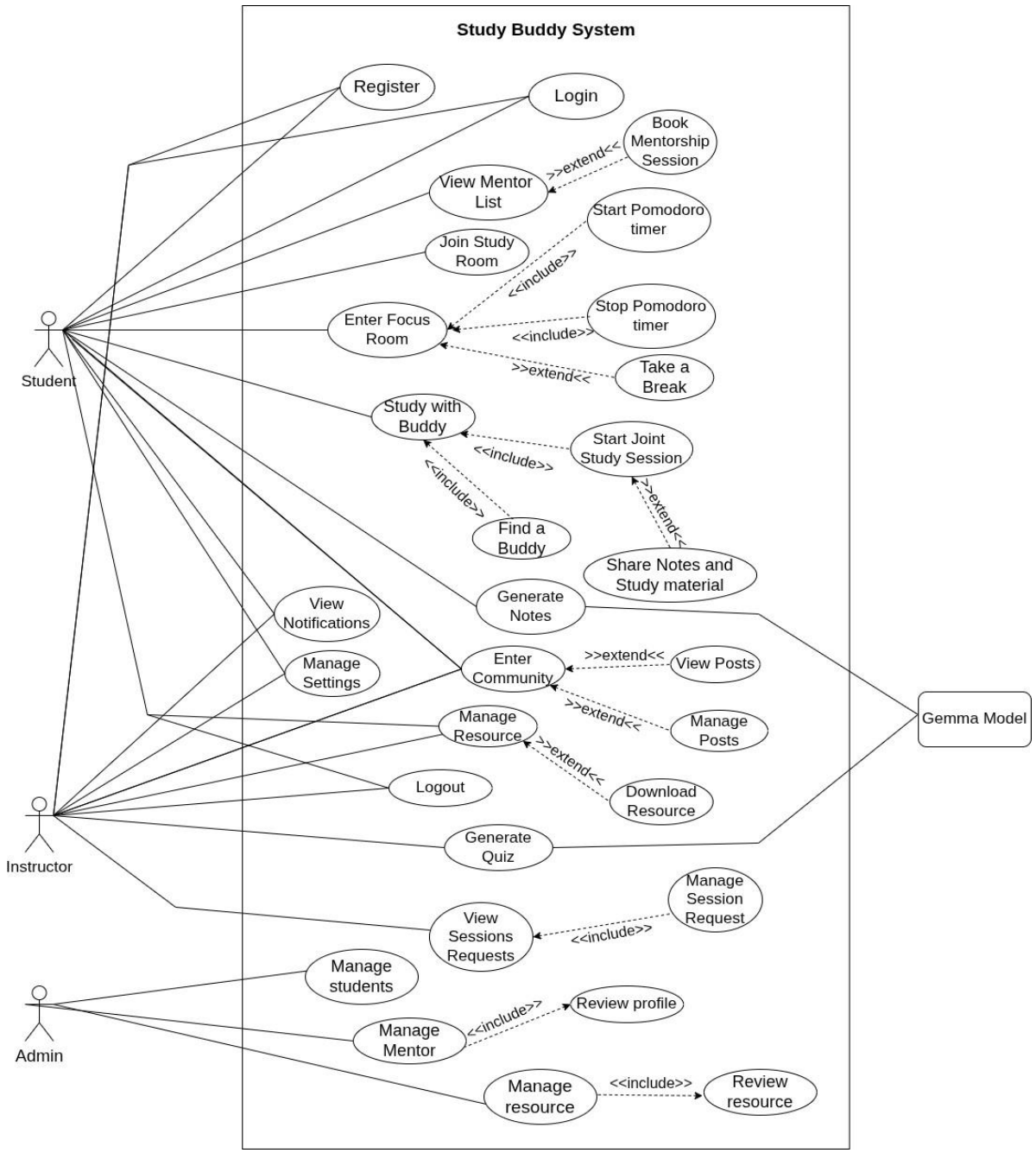


Figure 3.1: Use Case Diagram

3.2. Detailed Use Case

In detail use case, all of the pre conditions, the primary action, the alternative action(s), the post conditions, and exception handling for an entire interaction between an actor and a system is described.

UC-1: Register User

Table 3.1 details the use case Register User.

Table 3.1 UC-1 Register User

Use Case ID:	UC-1
Use Case Name:	Register
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	A new user signs up on the Study Buddy platform by providing personal details and selecting their role (Student or Instructor).
Trigger:	User selects “Register” on the login page.
Preconditions:	PRE-1: User has internet access. PRE-2: User has a valid email address.
Postconditions:	POST-1: User accounts are created and stored in the database. POST-2: User is redirected to the login page.
Normal Flow:	1. User clicks "Register." 2. System displays the registration form. 3. User enters required details (name, email, password, etc.). 4. System validates the input data (e.g., unique email). 5. System hashes the password and creates the user account. 6. System redirects users to login page for login.
Exceptions:	5.0 E1 Email Already Exists: System displays an error and prompts the user to log in or use a different email.
Business Rules:	BR-1: Email must be unique and validated upon registration. The system should securely hash and store the user's password and generate a JSON Web Token (JWT) if the password is successful during registration, and use this to secure access to the system afterward.
Assumptions:	The database (MongoDB) is operational for storing user data.

UC-2: Login

Table 3.2 details the use case Login Account.

Table 3.2 UC-2 Login Account

Use Case ID:	UC-2
Use Case Name:	Login
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	Allows an existing user to securely access their account using their registered credentials.
Trigger:	User opens the website.
Preconditions:	PRE-1: User is already registered. PRE-2. Internet connection is available.
Postconditions:	POST-1: User session is authenticated and established using JWT. POST-2: User is redirected to the personalized Dashboard.
Normal Flow:	1. User clicks "Log In." 2. System shows the sign in screen. 3. User types in the email and password to log in. 4. Checks credentials against database and returns an error message if they do not match. 5. If it's correct, System will create a JWT, and log the user in. 6 6. System redirects the user to their Dashboard.
Alternative Flows:	4.1 Incorrect Credentials: System displays an error of incorrect credentials and prompts the user to retry.
Exceptions:	4.0.E1 Login Service Unavailable: System notifies the user and advises them to try again later.
Business Rules:	BR-1: Access must be secure using JWT. BR-2: Both the email and password fields must be completed before the system attempts to process the login request (Required Fields)
Assumptions:	1. The user remembers their credentials.

UC-3: Manage Settings

Table 3.3 details the use case Manage Settings.

Table 3.3 UC-3 Manage Settings

Use Case ID:	**UC-3**
Use Case Name:	Manage Settings
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	Allows a logged-in user to update their personal information, notification preferences, and other account settings.
Trigger:	User selects "Manage Settings" from the profile menu.
Preconditions:	PRE-1: User is successfully logged in.
Postconditions:	POST-1: Updated account settings are saved and reflected in the user's profile.
Normal Flow:	1. User clicks "Manage Settings." 2. System shows the settings form (profile information, preferences, notifications). 3. User modifies the desired fields. 4. User clicks on Save changes 5. System validates the changes and updates the user record. 6. System confirms that settings have been saved.
Exceptions:	5.0.E1 Validation Failure: System rejects the change and highlights the error.
Business Rules:	BR-1: Password changes must require re-authentication.
Assumptions:	1. The settings updates are immediately synchronized across the system.

UC-4: Logout

Table 3.4 details the use case Logout.

Table 3.4 UC-4 Logout

Use Case ID:	UC-4
Use Case Name:	Logout
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	Allows a logged-in user to securely end their session on the platform.
Trigger:	User selects "Logout" from the profile menu.
Preconditions:	PRE-1: User is currently logged in.
Postconditions:	POST-1: The user's JWT session token is invalidated. POST-2: The user is redirected to the main login/home page.
Normal Flow:	1. User clicks "Logout." 2. System invalidates the current session and JWT. 3. System redirects the user to the log-in screen.
Exceptions:	E1: Session Invalidation Failure: System displays an error, but the user must close the browser/app to fully terminate the session.
Business Rules:	BR-1: Session termination must be immediate upon user request.
Assumptions:	1. No sensitive data is cached on the client side after logout.

UC-5: View Mentor List

Table 3.5 details the use case View Mentor List.

Table 3.5 UC-5 View Mentor List

Use Case ID:	UC-5
Use Case Name:	View Mentor List
Actors:	**Primary Actor:** Student. **Secondary Actors:** None
Description:	Allows a Student to browse the list of available Instructors/Mentors for academic help.
Trigger:	Student selects or "Mentorship " from the side bar.
Preconditions:	PRE-1: Student is logged in. PRE-2: Instructor profiles have been created and are marked as available.
Postconditions:	POST-1: A filtered list of available Mentors is displayed, showing their subjects, qualification and ratings.
Normal Flow:	1. Student navigates to the Mentorship module. 2. System queries the database for the list of Mentor. 3. They will be listed in the system and filtering and sorting options such as by subject will be provided 4. Student can click the List to see the Mentor profile details.
Business Rules:	BR-1: Mentors must set their subjects and availability before they appear on the list.
Assumptions:	1. The initial version relies on manual mentor management.

UC-6: Book Mentorship Session

Table 3.6 details the use case Book Mentorship Session.

Table 3.6 UC-6 Book Mentorship Session

Use Case ID:	UC-6
Use Case Name:	Book Mentorship Session
Actors:	**Primary Actor:** Student. **Secondary Actor:** None.
Description:	A Student selects a Mentor and requests a one-on-one or group session based on the Mentor's availability.
Trigger:	Student clicks "Book Mentor" or selects a Mentor profile.
Preconditions:	PRE-1: Student is logged in. PRE-2: Mentor is available for booking.
Postconditions:	POST-1: A session request is generated and sent to the Instructor. POST-2: The session status is set to "Pending Acceptance" on both dashboards.
Normal Flow:	1. Student chooses a Mentor from the list. 2. System displays the Mentor's availability. 3. Student chooses a time period in which they are available. 4. Student agrees to the booking and submits a request. 5. System sends a request to the Instructor. 6. Continues with the Accept/Reject Session Request flow.
Business Rules:	BR-1: Scheduling is handled manually in the initial phase.
Assumptions:	1. The Mentor actively reviews session requests.

UC-7: View Sessions Requests

Table 3.7 details the use case View Sessions Requests.

Table 3.7 UC-7 View Sessions Requests

Use Case ID:	UC-7
Use Case Name:	View Sessions Requests
Actors:	**Primary Actor:** Instructor. **Secondary Actors:** None
Description:	Allows an Instructor to view all pending requests for mentorship sessions from Students.
Trigger:	Instructor selects "View Sessions Requests" from their dashboard/menu.
Preconditions:	PRE-1: Instructor is logged in. PRE-2: One or more session requests have been submitted by Students.
Postconditions:	POST-1: A list of session requests is displayed for the instructor to review.
Normal Flow:	1. Instructor navigates to the request management panel. 2. System fetches and presents the list of all the student's pending session requests along with Student name, desired session time and subject 3. Instructor reviews the details of a specific request. 4. Continues with the Accept Session Request or Reject Session Request flow.
Business Rules:	BR-1: Session requests must be displayed in chronological order.
Assumptions:	1. The Instructor's schedule is up-to-date.

UC-8: Manage Session Request

Table 3.8 details the use case Manage Session Requests.

Table 3.8 UC-8 Manage Session Request

Use Case ID:	UC-8
Use Case Name:	Manage Session Request
Actors:	**Primary Actor:** Instructor. **Secondary Actors:** None
Description:	An included flow where the Instructor formally accepts or rejects a session request made by a Student.
Trigger:	Instructor selects "Accept" or “Rejects” on a pending session request.
Preconditions:	PRE-1. Instructor is logged into the Study Buddy System. PRE-2: Instructor is viewing a pending session request.
Postconditions:	POST-A1. Session status is updated to "Confirmed." or “Rejected” POST-A3. Student receives a final confirmation notification about acceptance or rejection of the Session requests.
Normal Flow:	1. Instructor selects "Accept" on a request. 2. System verifies the time slot is still free. 3. System updates the session status to "Confirmed" and logs the action. 4. System sends confirmation notifications to the Student and Instructor.
Business Rules:	BR-1: Acceptance must update the Mentor's availability for that time slot.
Assumptions:	1. Payment verification must be done before accepting the session request.

UC-9: Enter Focus Room

Table 3.9 details the use case Enter Focus Room.

Table 3.9 UC-9 Enter Focus Room

Use Case ID:	UC-9
Use Case Name:	Enter Focus Room
Actors:	**Primary Actor:** Student. **Secondary Actors:** None
Description:	The Student accesses the distraction-free environment to study individually using productivity techniques.
Trigger:	Student selects "Focus Room" from the menu.
Preconditions:	PRE-1: Student is logged in.
Postconditions:	POST-1: The Focus Room interface is displayed. POST-2: The Start/Stop Pomodoro Timer flow is initialized.
Normal Flow:	1. Student clicks "Enter Focus Room." 2. System loads the minimalist, distraction-free environment. 3. System displays the options to start the Pomodoro timer and the Take a Break feature. 4. Continues with the Start/Stop Pomodoro Timer flow.
Business Rules:	BR-1: Focus Room interface must minimize distractions.
Assumptions:	1. The user understands the concept of the Pomodoro technique.

UC-10: Start/Stop Pomodoro Timer

Table 3.10 details the use case Start/Stop Pomodoro Timer.

Table 3.10 UC-10 Start/Stop Pomodoro Timer

Use Case ID:	UC-10
Use Case Name:	Start/Stop Pomodoro Timer
Actors:	**Primary Actor:** Student. **Secondary Actors:** None
Description:	An included flow where the Student uses the Pomodoro timer to enforce structured study intervals.
Trigger:	Student clicks "Start timer" in the Focus Room.
Preconditions:	PRE-1: Student is logged in. PRE-1: Student is in the Focus Room.
Postconditions:	POST-1: After 1 timer expires the student can take a break. POST-2: Student can start the new work cycle.
Normal Flow:	1. Student clicks "Start." 2. System starts the work timer (e.g., 25 minutes). 3. Timer expires; System plays a notification. 4. Continues with the Take a Break flow or starts the next work cycle. 5. Student clicks "Stop" to end the session.
Business Rules:	BR-1: The technique helps maintain consistency and focus.

UC-11: Take a Break

Table 3.11 details the use case Take a Break.

Table 3.11 UC-11 Take a Break

Use Case ID:	UC-11
Use Case Name:	Take a Break
Actors:	**Primary Actor:** Student. **Secondary Actors:** None
Description:	An extended flow (optional action) that starts a scheduled rest period after a Pomodoro work interval is complete.
Trigger:	Student clicks "Take a Break" after a work timer expires.
Preconditions:	PRE-1: The Pomodoro work timer has expired.
Postconditions:	POST-1: The break duration is logged, or the student is prompted to return to work.
Normal Flow:	1. Work timer expires. 2. Student selects "Take a Break." 3. System starts the break timer (e.g., 5 minutes). 4. Break timer expires, System notifies the Student. 5. System prompts the Student to resume the study session.
Business Rules:	BR-1: Breaks must adhere to the standard Pomodoro intervals.
Assumptions:	1. The break timer is displayed clearly.

UC-12: Find a Buddy

Table 3.12 details the use case Find a Buddy.

Table 3.12 UC-12 Find a Buddy

Use Case ID:	UC-12
Use Case Name:	Find a Buddy
Actors:	**Primary Actor:** Student. **Secondary Actors:** None
Description:	The Student searches for and identifies peers who have similar study interests to initiate a collaborative session.
Trigger:	Student selects "Study with Buddy" from the menu.
Preconditions:	PRE-1: Student is logged in. PRE-2: Student has set their study preferences/subjects.
Postconditions:	POST-1: A list of matched peers is displayed.
Normal Flow:	1. Student selects "Find a Buddy." 2. System prompts the Student to select a subject/topic. 3. System runs the matching algorithm to filter online peers. 4. System displays suggested peers. 5. Continues with the Study with Buddy flow.
Business Rules:	BR-1: Matching is based on similar topics/subjects.
Assumptions:	1. Peers listed are currently available for a session.

UC-13: Study with Buddy

Table 3.13 details the use case Study with Buddy.

Table 3.13 UC-13 Study with Buddy

Use Case ID:	UC-13
Use Case Name:	Study with Buddy
Actors:	**Primary Actor:** Student. **Secondary Actors:** None
Description:	The Student initiates a real-time study session with a peer found through the matching system.
Trigger:	Student selects a matched peer and sends a session request.
Preconditions:	PRE-1: Peer matching (UC-14) is successful. PRE-2: The selected peer is available.
Postconditions:	POST-1: A joint study session (Study Room) is created. POST-2: Students are connected via real-time comms.
Normal Flow:	1. Student sends a session request to a matched peer. 2. Peer accepts the request. 3. System executes the Start Joint Study Session flow. 4. Students can communicate via chat or video.
Business Rules:	BR-1: Sessions must support real-time communication.
Assumptions:	1. The students' devices support video/audio comms.

UC-14: Start Joint Study Session

Table 3.14 details the use case Start Joint Study Session.

Table 3.14 UC-14 Start Joint Study Session

Use Case ID:	UC-14
Use Case Name:	Start Joint Study Session
Actors:	**Primary Actor:** Student (Implicit in Diagram). **Secondary Actors:** None
Description:	An **included** flow where the System establishes the real-time, collaborative environment for matched students.
Trigger:	A peer accepts the "Study with Buddy" request.
Preconditions:	PRE-1: Mutual consent for the session is established. PRE-2: WebRTC/Agora SDK services are running.
Postconditions:	POST-1: A virtual study room is initialized. POST-2: The session time starts logging.
Normal Flow:	1. System validates peer connection status. 2. System initiates the WebRTC/Agora connection for video/audio. 3. System loads the collaborative study room interface. 4. System executes the Share Notes and Study Material flow. 5. Students can now interact.
Business Rules:	BR-1: Session establishment must be low latency.

UC-15: Share Notes and Study Material

The following Table 3.15 gives the description of the use case Share Notes and Study Material.

Table 3.15 UC-15 Share Notes and Study Material

Use Case ID:	UC-15
Use Case Name:	Share Notes and Study Material
Actors:	**Primary Actor:** Student. **Secondary Actors:** None
Description:	An included flow that allows students in a joint study session to exchange digital resources.
Trigger:	Student clicks the "Share" button in the joint study session interface.
Preconditions:	PRE-1: Students are in a joint study session (UC-16).
Postconditions:	POST-1: The shared material is accessible to all students in the room.
Normal Flow:	1. Student selects "Share Material." 2. Student uploads/selects a file (notes, quiz etc.). 3. System uploads the file to temporary session storage. 4. System notifies the other peer(s) that a file has been shared. 5. Other peer(s) can view/download the file.
Business Rules:	BR-1: Shared resources must be viewable within the study room interface.

UC-16: Download Resource

The following table 3.16 represents fully dressed use case of Download Resource.

Table 3.16 UC-16 Download Resource

Use Case ID:	UC-16
Use Case Name:	Download Resource
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	An extended flow where the user obtains a copy of a resource from the Resource Hub.
Trigger:	User clicks "Download" on a resource.
Preconditions:	PRE-1: The resource is available for download.
Postconditions:	POST-1: The resource file transfer is initiated on the user's device.
Normal Flow:	1. User selects a resource. 2. User clicks "Download." 3. System initiates the file transfer to the user's device. 4. System logs the download.
Business Rules:	BR-1: Requires payment/purchase confirmation for non-free resources(in Future).
Assumptions:	1. The user's device has storage space.

UC-17: Generate Notes

Table 3.17 details the use case Generate Notes.

Table 3.17 UC-17 Generate Notes

Use Case ID:	UC-17
Use Case Name:	Generate Notes
Actors:	**Primary Actor:** Student. **Secondary Actors:** Gemma Model (AI Integration).
Description:	A Student uses AI through integrated APIs to automatically generate notes, summaries, or quizzes from uploaded study material.
Trigger:	Student selects "Content Generator" from the dashboard or menu.
Preconditions:	PRE-1: Student is logged in. PRE-2: Student has given a command or a topic to the chat to generate notes. PRE-3: The Gemma model is working.
Postconditions:	POST-1: AI-generated content (notes/quiz/summary) is displayed. POST-2: Content is saved in the chat until the student got logged out.
Normal Flow:	1. Student navigates to the Content Generator. 2. Student gives a command or a topic to the chat to generate notes. 3. System sends the content to the Gemma model. 4. Gemma Model processes the request. 5. System receives the generated text. 6. System displays the output for review.
Exceptions:	4.0.E1 AI Service Unavailable: System displays an error and advises the student to try again later.
Business Rules:	BR-1: Content generation accuracy depends on the quality of the uploaded material.
Assumptions:	1. The question asked should be according to limited knowledge of the gemma model.

UC-18: Enter Community

Table 3.18 details the use case Enter Community.

Table 3.18 UC-18 Enter Community

Use Case ID:	UC-18
Use Case Name:	Enter Community
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	The user accesses the forum module to view discussions, post questions, and engage with the online learning community.
Trigger:	User selects "Community" from the menu.
Preconditions:	PRE-1: User is logged in.
Postconditions:	POST-1: The community forum feed is displayed. POST-2: The View Posts flow is initialized.
Normal Flow:	1. User clicks "Enter Community." 2. System loads the main forum feed, typically displaying recent posts. 3. User can scroll to view current discussions. 4. Continues with the View Posts flow or the Manage Posts flow.
Business Rules:	BR-1: The module encourages engagement and peer support.
Assumptions:	1. Posts are categorized typically by recent posts.

UC-19: View Posts

Table 3.19 details the use case View Posts.

Table 3.19 UC-19 View Posts

Use Case ID:	UC-19
Use Case Name:	View Posts
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	An extended flow where the user browses and reads posts created by other users in the community.
Trigger:	User scrolls through the community feed or search for a topic.
Preconditions:	PRE-1: User is in the Community module.
Postconditions:	POST-1: The content of the selected post, including comments, is displayed.
Normal Flow:	1. User scrolls through the main feed. 2. User selects a post title/card. 3. System loads the full post view, including the discussion thread (comments). 4. User can optionally contribute a comment or like the post.
Business Rules:	BR-1: Posts should be easily searchable and filterable by topic.
Assumptions:	1. Posts are loaded quickly from the database.

UC-20: Manage Posts

The following table 3.20 represents fully dressed use case of Manage Posts.

Table 3.20 UC-20 Manage Posts

Use Case ID:	UC-20
Use Case Name:	Manage Posts
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	An extended flow where the user creates, edits, or deletes their own content (posts and comments) in the forum.
Trigger:	User selects "Create New Post" or clicks "Edit/Delete" on their existing post.
Preconditions:	PRE-1: User is in the Community module.
Postconditions:	POST-1: New post is published, or existing post is updated/removed.
Normal Flow:	1. User selects "Create New Post." 2. User enters text, tags, and media. 3. User clicks "Publish." 4. System validates and saves the post. (Alternative: User clicks "Edit" on their post, updates content, and saves.)
Business Rules:	BR-1: Users can only edit or delete their own posts.
Assumptions:	1. The community is not used for any other content except the relative one.

UC-21: Manage Resource

The following table 3.21 represents fully dressed use case of Manage Resource.

Table 3.21 UC-21 Manage Resource

Use Case ID:	UC-21
Use Case Name:	Manage Resource
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	The user interacts with the Resource Hub by uploading resources for sharing or by downloading resources created by others.
Trigger:	User selects "Resource Hub" from the menu.
Preconditions:	PRE-1: User is logged in. PRE-1: User is in Resource Hub.
Postconditions:	POST-1: The user successfully interacts with the resource (e.g., uploads a new file, starts a download).
Normal Flow (Browse/Download):	1. User navigates to the Resource Hub. 2. User searches or filters for study materials (notes, quizzes, guides). 3. User selects a resource. 4. System executes the Download Resource flow.
Normal Flow (Upload/Share):	1. User clicks "Upload". 2. User provides metadata and the resource file. 3. System processes and lists the resource.
Business Rules:	BR-1: Allows users to share and download high-quality educational resources. BR-2: Paid Resources are not integrated in the current version.
Assumptions:	1. Resource quality is subject to user review and rating.

UC-22: View Notifications

The following table 3.22 represents fully dressed use case of View Notifications.

Table 3.22 UC-22 View Notifications

Use Case ID:	UC-22
Use Case Name:	View Notifications
Actors:	**Primary Actor:** Student, Instructor. **Secondary Actors:** None
Description:	The user receives real-time alerts for important events such as new study matches, session confirmations, or community updates.
Trigger:	System generates an alert for a relevant event.
Preconditions:	PRE-1: User is logged in. PRE-2: User has enabled notifications in settings.
Postconditions:	POST-1: The user is informed of the event. POST-2: The notification is logged as read when viewed.
Normal Flow:	1. An event occurs (e.g., a peer accepts a study request). 2. A push notification is sent to users' devices via system. 3. User clicks on the notification (such as the Notification bell icon). 4. System will mark the message as read.
Business Rules:	BR-1: Notifications must keep users informed and engaged.
Assumptions:	1. Device tokens are valid for sending push notifications.

UC-23: Manage Mentor

The following table 3.23 represents fully dressed use case of Manage Mentor.

Table 3.23 UC-23 Manage Mentor

Use Case ID:	UC-23
Use Case Name	Manage Mentor
Actors	Primary Actor: Admin. Secondary Actors: None
Description	Gives the Admin the ability to see a list of all of the registered mentors on the platform, verify their profile for authenticity, block and unblock mentor profiles when needed and accept additional requests.
Trigger	Admin selects "Manage Mentors" from the Admin Dashboard.
Preconditions	PRE-1: Admin is successfully logged in. PRE-2: At least one mentor account exists in the system.
Postconditions	POST-1: Mentor verification status is updated and reflected across the platform. POST-2: Blocked mentors lose access to all platform features until unblocked.

UC-24: Manage Students

The following table 3.24 represents fully dressed use case of Manage Students.

Table 3.24 UC-24 Manage Students

Use Case ID:	UC-24
Use Case Name	Manage Students
Actors	Primary Actor: Admin. Secondary Actors: None
Description	Allows the Admin to view all registered student accounts, monitor their activity status, and block or unblock student accounts in response to policy violations or administrative decisions.
Trigger	Admin selects "Manage Students" from the Admin Dashboard.
Preconditions	PRE-1: Admin is successfully logged in. PRE-2: At least one student account exists in the system.
Postconditions	POST-1: Student account status is updated and reflected immediately across the platform. POST-2: Blocked students are denied access to all platform features until the Admin reverses the action.

3.3. Functional Requirements

This section lists the core functions that the StudyBuddy must perform, detailing the specific features and behaviours required to fulfil the needs of users and stakeholders.

User Authentication:

As shown in Table 3.25, FR-1 User Authentication is mapped to its design component.

Table 3.25 FR-1 User Authentication

Identifier	**FR-1**
Title	User Authentication
Requirement	The Student or Instructor shall be able to securely register, log in, and log out using email and password. The system shall authenticate users using JWT-based security.
Source	System Stakeholders, Use Cases UC-1, UC-2, UC-4
Rationale	To ensure secure access to system features.
Business Rule	Email must be unique; password must be hashed.
Dependencies	FR-2
Priority	High

Profile & Account Management:

As shown in Table 3.26, FR-2 Profile & Account Management is mapped to its design component.

Table 3.26 FR-2 Profile & Account Management

Identifier	**FR-2**
Title	Profile & Account Management
Requirement	The user shall be able to update profile details such as name, password, preferences, and notification settings.
Source	UC-3
Rationale	Users require control over personal information.
Business Rule	Password changes require re-authentication.
Dependencies	FR-1
Priority	Medium

View Mentor List:

As shown in Table 3.27, FR-3 View Mentor List is mapped to its design component.

Table 3.27 FR-3 View Mentor List

Identifier	**FR-3**
Title	View Mentor List
Requirement	The Student shall be able to view a list of available Mentors including subjects, qualifications, and availability.
Source	UC-5
Rationale	Students need access to suitable mentors.
Business Rule	Mentors must mark availability to be shown.
Dependencies	FR-1
Priority	High

Book Mentorship Session:

As shown in Table 3.28, FR-4 Book Mentorship Session is mapped to its design component.

Table 3.28 FR-4 Book Mentorship Session

Identifier	FR-4
Title	Book Mentorship Session
Requirement	The Student shall be able to request and schedule a mentorship session by selecting a Mentor and choosing a time slot.
Source	UC-6
Rationale	To enable structured guidance sessions.
Business Rule	Scheduling is managed manually in MVP.
Dependencies	FR-3, FR-5
Priority	High

Manage Mentorship Requests

As shown in Table 3.29, FR-5 Manage Mentorship Requests is mapped to its design component.

Table 3.29 FR-5 Manage Mentorship Requests

Identifier	FR-5
Title	Manage Mentorship Requests
Requirement	The Mentor shall be able to view, accept, or reject session requests from Students. The system shall notify Students of the decision.
Source	UC-7, UC-8
Rationale	Mentors must control their session schedule.
Business Rule	Accepted sessions must block the time slot.
Dependencies	FR-4
Priority	High

Peer Matching (Find a Buddy)

As shown in Table 3.30, FR-6 Peer Matching (Find a Buddy) is mapped to its design component.

Table 3.30 FR-6 Peer Matching (Find a Buddy)

Identifier	FR-6
Title	Peer Matching
Requirement	The Student shall be able to find study partners based on matched subjects or study preferences.
Source	UC-12
Rationale	Enables collaborative study.
Business Rule	Matching is based on shared interests/subjects.
Dependencies	FR-1
Priority	High

Real-Time Study Session

As shown in Table 3.31, FR-7 Real-Time Study Session is mapped to its design component.

Table 3.31 FR-7 Real-Time Study Session

Identifier	FR-7
Title	Real-Time Study Session
Requirement	The system shall allow matched Students to initiate a real-time video/audio study session using WebRTC/Agora.
Source	UC-13, UC-14
Rationale	Supports live peer-to-peer study.
Business Rule	Session must allow low-latency communication.
Dependencies	FR-6
Priority	High

AI-Based Content Generator

As shown in Table 3.32, FR-8 AI-Based Content Generator is mapped to its design component.

Table 3.32 FR-8 AI-Based Content Generator

Identifier	FR-8
Title	AI-Based Content Generator
Requirement	The Student shall be able to generate notes, summaries, or quizzes using integrated AI (Gemma/DeepSeek) by providing a topic or study content.
Source	UC-16
Rationale	Helps students study efficiently.
Business Rule	Accuracy depends on input quality.
Dependencies	System Speed.
Priority	High

Focus Room with Pomodoro Timer

As shown in Table 3.33, FR-9 Focus Room with Pomodoro Timer is mapped to its design component.

Table 3.33 FR-9 Focus Room with Pomodoro Timer

Identifier	FR-9
Title	Focus Room / Pomodoro Timer
Requirement	The system shall provide a distraction-free Focus Room where Students can start/stop a Pomodoro timer and take scheduled breaks.
Source	UC-9, UC-10, UC-11
Rationale	Supports focused study habits.
Business Rule	Break and timer intervals follow Pomodoro rules.
Dependencies	FR-1
Priority	Medium

Community Forum & Resource Hub

As shown in Table 3.34, FR-10 Community Forum & Resource Hub is mapped to its design component.

Table 3.34 FR-10 Community Forum & Resource Hub

Identifier	FR-10
Title	Community Forum & Resource Hub
Requirement	The system shall allow users to participate in community discussions (view, create, edit posts) and upload/download study resources through the Resource Hub.
Source	UC-17 to UC-21
Rationale	Encourages community learning and resource sharing.
Business Rule	Users can modify only their own posts; paid resources not supported in MVP.
Dependencies	FR-1
Priority	Medium

3.4. Non-Functional Requirements

This section specifies the quality attributes of the TrackMate including usability, performance, security, reliability, and other criteria that ensure a robust and user-friendly system

3.4.1. Usability

Usability requirements include learning, simplicity of interaction, error prevention and accessibility. All the requirements ensure StudyBuddy provides a seamless and intuitive user experience for all its modules.

- **USE-1:** A new user shall be able to register and proceed to their StudyBuddy Dashboard within a maximum of 3 minutes after accessing the system for the first time.
- **USE-2:** The system shall provide clear navigation menus, tooltips, and prompts to guide first-time users throughout mentorship booking, content generation, and study sessions.
- **USE-3:** The interface of the Focus Room shall remain distraction-free with minimal UI elements to promote concentration.

3.4.2. Performance

These shall specify measurable performance requirements for load, content generation, and real-time user interaction.

- **PER-1:** 95% of StudyBuddy web pages shall load completely within **3 seconds** on a 20 Mbps internet connection.
- **PER-2:** The AI Content Generator shall generate notes and summaries of at least 1,000 words without significant delay or crash under normal server load in its MVP version.
- **PER-3:** Under normal server load in its MVP version, the system shall support at least 100 simultaneous active Internet users without significant delay or crash.

3.4.3. Reliability

Reliability makes sure that the system is positive and steady without failures, which is involved in providing accurate performances.

- **REL-1:** The StudyBuddy backend shall maintain 99% uptime monthly, excluding scheduled maintenance.
- **REL-3:** In case of AI service failure (Gemma), the system shall display an error.
- **REL-4:** The system shall prevent data loss by ensuring that mentorship bookings, posts, and shared resources are saved atomically.

3.4.4. Security

Security secures user data and system resources from unauthorized breaches, use and access.

- **SEC-1:** All passwords shall be hashed using bcrypt or stronger hashing algorithms before being stored in the database.
- **SEC-2:** All API requests to protected routes shall require JWT-based authorization.
- **SEC-3:** Real-time communication in study sessions shall be encrypted using WebRTC secure protocols.

3.4.5. Portability

These requirements are necessary to allow the system to operate with efficiency in a number of today's operating environments and their ability to add on easily.

- **PORT-1:** The StudyBuddy platform shall provide identical core functionality across all modern browsers (Chrome, Edge, Firefox, Safari).
- **PORT-2:** The backend APIs shall be designed to allow future integration with Android/iOS mobile apps without major architectural changes.

4. Design and Architecture

In this chapter, we will be defining the structure of the StudyBuddy Web Application. The physical and logical division of the system is drawn and the communication among independent modules are shown. With design freezes, visualization is separated from the data collection, which can be done at high frequency without compromise to the user interface's responsiveness.

4.1. System Architecture

The Study Buddy system will be implemented using the Three-Tier Layered Architecture: Presentation Layer, Application/Business Logic Layer, and Data Layer. The benefits include separation of concerns, scalability, maintainability, and modular addition to the platform.

4.1.1. Presentation Layer (Frontend – React/Tailwind)

The Presentation Layer is to be done using React and Tailwind CSS so that the UI can be responsive. This layer provides all user interactions, dashboard views, mentorship booking interfaces, Focus room solutions, AI content generation UI, community forums, resource hub features and more.

They securely communicate with the back-end APIs through HTTPS and handle client state management, routing and session management. Real-time supports like Study-with-Buddy sessions are built in real-time in the browser through WebRTC/Agora client SDKs.

4.1.2. Application /Business Logic Layer (Backend – Node.js / Express)

The Application Layer is implemented using Node.js with Express.js and acts as the core processing unit of the system.
It handles:

- Request validation & authorization
- Session management
- Business logic for mentorship workflows, peer matching, community posts, and content generation
- Integration with external AI services (Ollama)
- Real-time signaling through WebSockets for peer and mentor interactions
- Booking management, notifications, and content upload handling

4.1.3. Data Layer (MongoDB)

Stores and retrieves persistent system data, and the AI model for generating quiz and notes across the generations.
This includes:

- User profiles and roles
- Mentor availability and mentorship sessions
- Peer-matching metadata
- Community posts

- Resource Hub file metadata
- AI-generated notes stored temporarily
- AI-generated quizzes
- Notifications and logs

4.1.4. Inter-Layer Relationships

Relationships among the Three Levels of Architecture for StudyBuddy are explained below:

Presentation Layer → Application Layer: The React-based front-end interfaces with the Application Layer mainly via secure HTTPS REST calls. All the users interact with each other, ranging from signing in to editing their mentor profiles or creating study content, which is all broken down into a structured HTTP request fired to the Express.js API. The Presentation Layer has web socket persistent connections, enabled by Socket.io, for live study room chat and peer-to-peer signaling interactions. Currently notifications are retrieved from REST endpoints but it can be extended to support future real-time pushes over the existing WebSocket layer.

Application Layer → Data Layer: This is the only link between the user interface and all data storage, the Application Layer. Controllers listen to requests and validate them with JWT, apply role based access control (RBAC) and pass business logic to other service modules. These services directly communicate with the Data Layer, which then uses it to manipulate the collections in MongoDB that contain data for entities like user profiles, mentorship sessions, quizzes, and notification logs. As of now, a number of the file-based information like mentor level documentation, user avatars etc. are being managed as base64 encoded strings and being stored directly in MongoDB documents to ensure high data locality and easy initial infrastructure needs.

Application Layer → AI Integration: If the student or instructor requests the generation of a piece of content (e.g., a quiz or study summary), the Application Layer's Ollama Service builds a prompt and sends it to an Ollama instance that is running the Ollama gemma 3 language model. This service is operational as an external product of data. The response generated is returned to the Application Layer and stored in the corresponding collection in the MongoDB for persistence and delivered back to the Presentation Layer for presentation to the user.

Application Layer → Real-Time Communication: Application Layer is used as a signaling server for Collaborative Study Rooms and Mentorship Scheduling sessions in Real-Time Communication. It sends WebRTC signaling information (offers, answers, and ICE candidates) to the participants through Socket.io. This allows the Presentation Layer to create direct, low-latency audio and video streams, with the media not going through the Application Layer. The Data Layer stores the session metadata, participants, and room state during the session's lifespan.

Presentation Layer → External Identity Provider: Google OAuth 2.0 authentication: The flow is started directly from the Presentation Layer to the Google identity platform. If the login is successful, the Application Layer receives the identity token generated. The Auth Service validates this token and resolves the user identity against the Data Layer (creating a record if needed), and then issues a platform-specific JWT. All subsequent authorised interactions on the architectural layers are governed by this JWT.

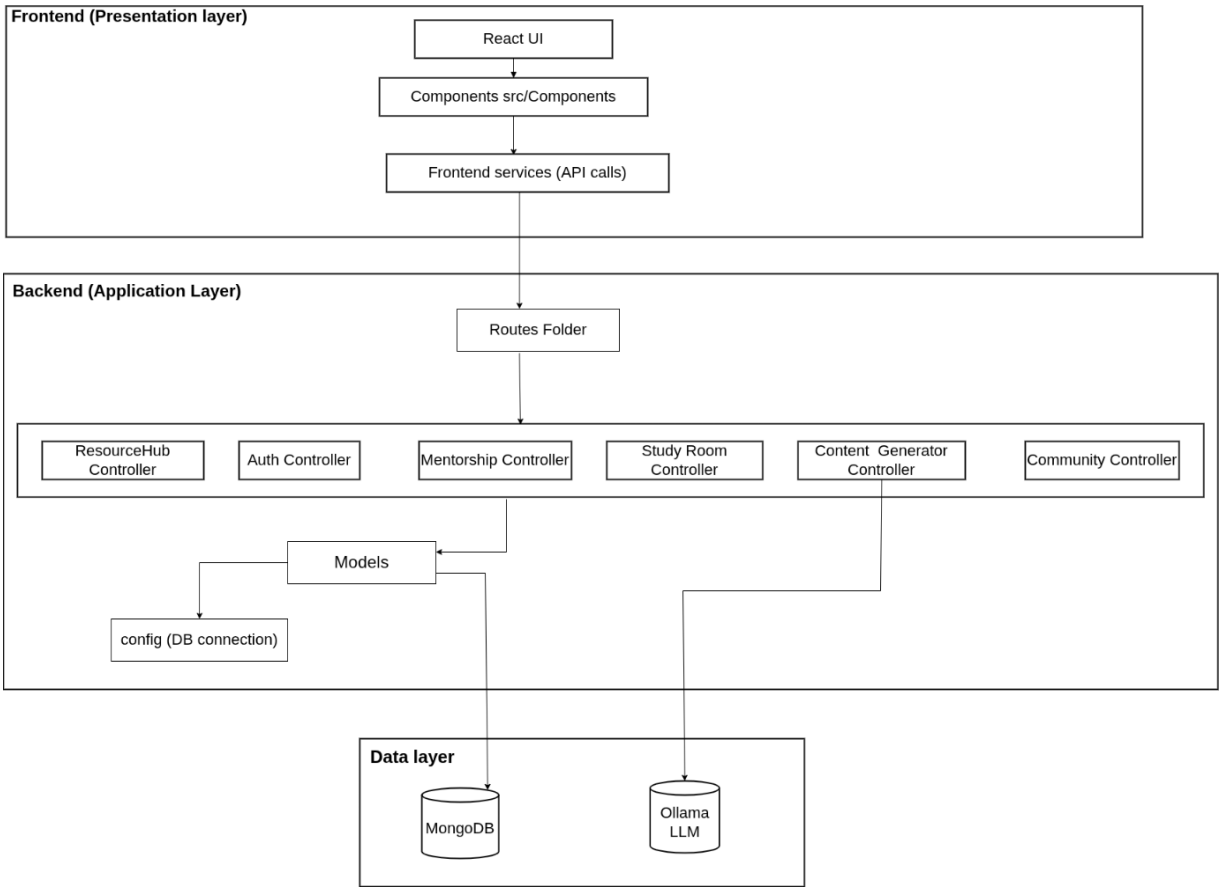


Figure 4.1 Architecture Diagram

4.2. Data Representation

To achieve a compromise between flexibility and scalability, StudyBuddy's data model is built around a document-oriented approach. The data is stored in the central datastore, MongoDB, where information is structured into Collections: users, mentorshipSessions, studyRooms, posts and resources. Large number of references are associated to user documents, which act as the central object, including links to other mentorship sessions (as a student and as an instructor), and to uploaded resources, authored posts. The system allows for openings for dynamic relationships (e.g., it is capable of including more than one student in the study rooms and allows for both association orientations when executing mentorship sessions at the same time). Content-driven collections, which include posts and resources, carry metadata like tags, engagement numbers and file references, making it easy to retrieve and manage interactions in the community. This schema design allows StudyBuddy to easily manage related pieces of academic activity and offer high-quality collaborative editing along with content access.

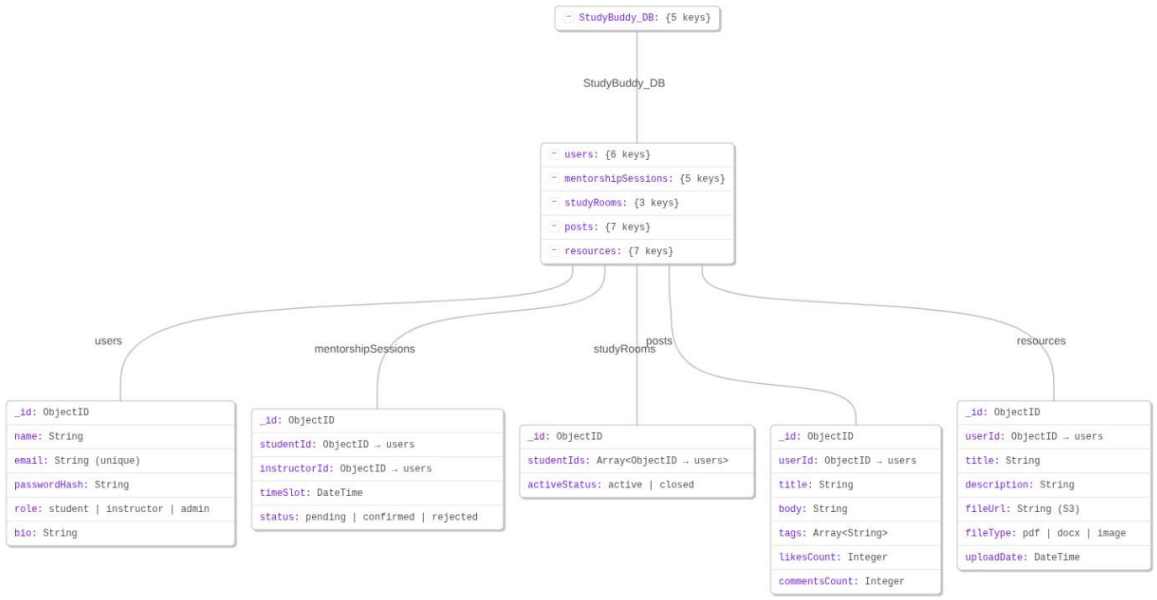


Figure 4.2 Json Tree

4.3. Process Flow/Representation

User traverses a process through an App, followed by an Auth – Dashboard – Feature Flow – Logout. These features will be realized as individual sub-flows, with defined entry paths and exit criteria.Each feature (Content Generator, Study Room, Mentorship, Resource hub, Community) will be realized as an independent sub-flow with a clear path of success and failure. Service calls to external systems (LLM, WebRTC/Agora, S3, MongoDB) are drawn as interactions with the external systems, including request/response arrows, and optional caching/rate-limit arrows. Utilize swimlanes to support Client, Controller, Service, Model/External responsibilities and annotate important decision points and error handlers. Add sequence numbers and brief notes for each step for developers to easily transform flows into controller procedures or service workflows.

I. **Student Activity Diagram**
   The Activity diagram depicts the path the student would take from opening Study Buddy to registering/login to the student dashboard to entry of feature flows such as Content Generator, Focus Room, Study Room, Community, Resource Hub, Study with Buddy, Mentorship. The diagram ends with logout.

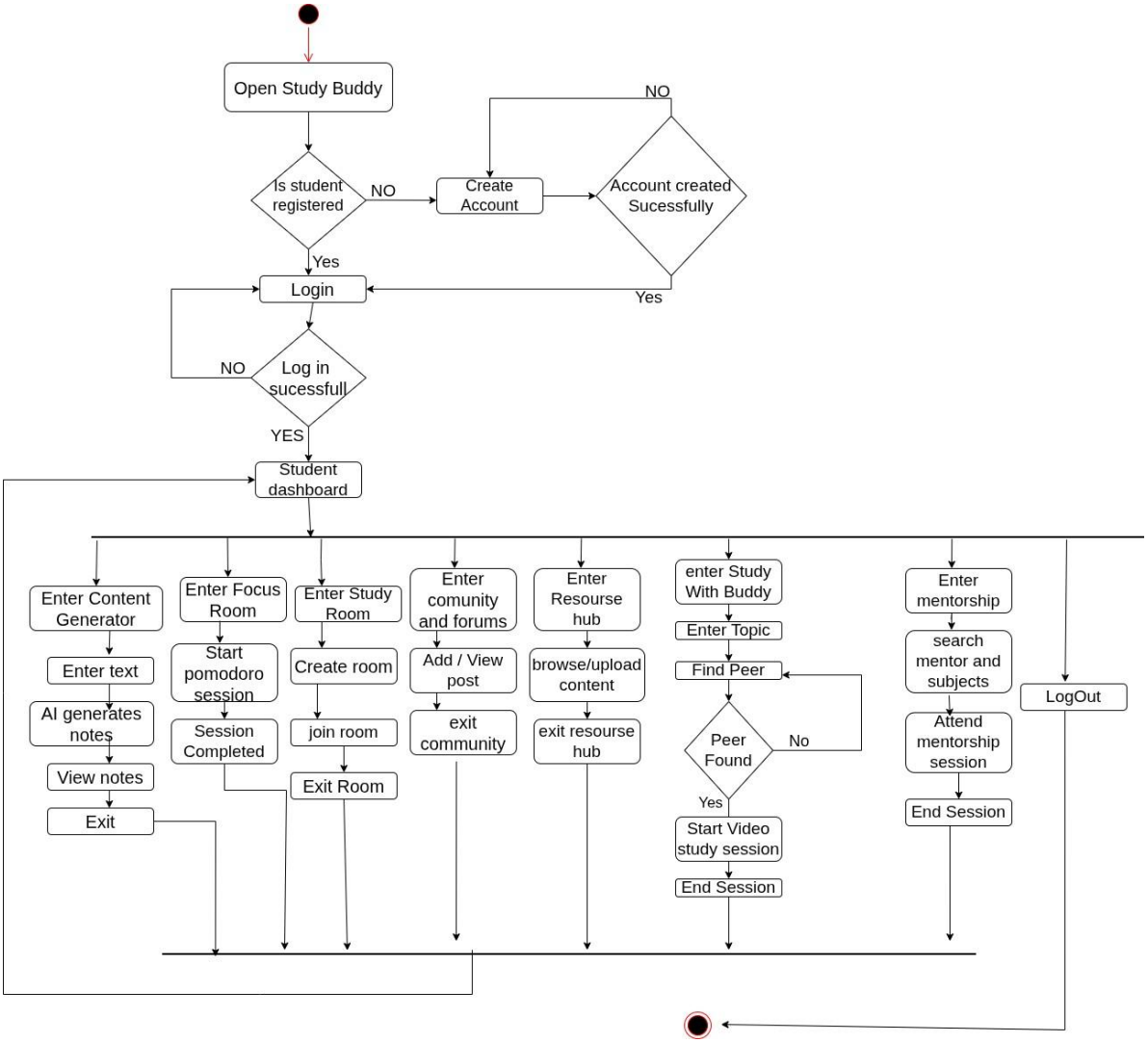


Figure 4.3 Process Flow/Student Activity Diagram

II. **Teacher Activity Diagram**

The activity diagram shows all activities of a Teacher/Instructor in the Study Buddy platform. The flow begins with Login and extends to managing their Availability, managing Session Requests (Accept/Reject), conducting Live Sessions, tracking Student Details, and managing Resource Hub and Community forums.

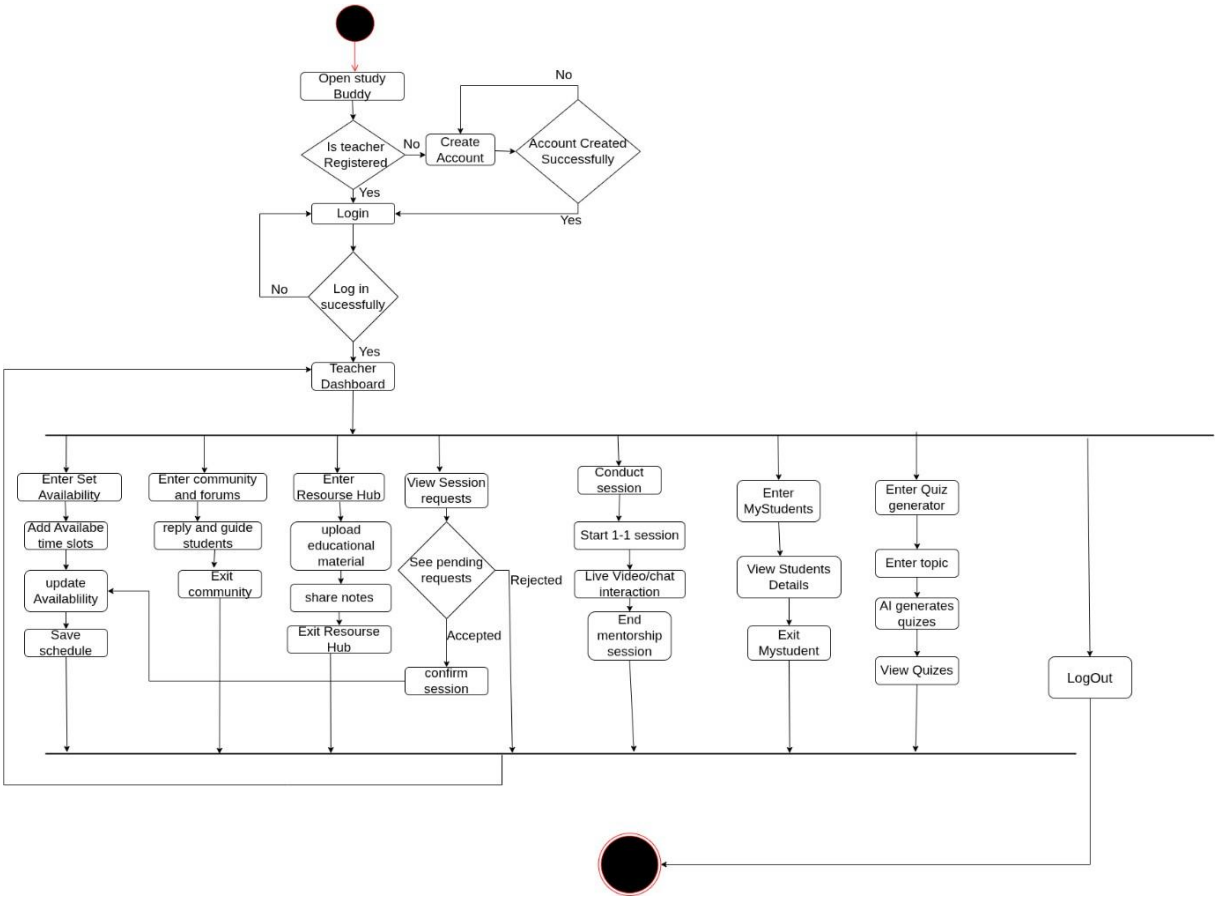


Figure 4.4 Process Flow/Teacher Activity Diagram

III. Admin Activity Diagram

The activity diagram shows the entire process for an Admin in StudyBuddy. It starts with Login and continues with user account management, instructor profile verification and monitoring of platform activity. The admin resolves reported problems, posts Community resources and posts and moderates community posts as necessary, and conducts mentorship sessions as needed. In addition, the admin user can check system analytics and has continuous supervision and control of the integrity of the platform.

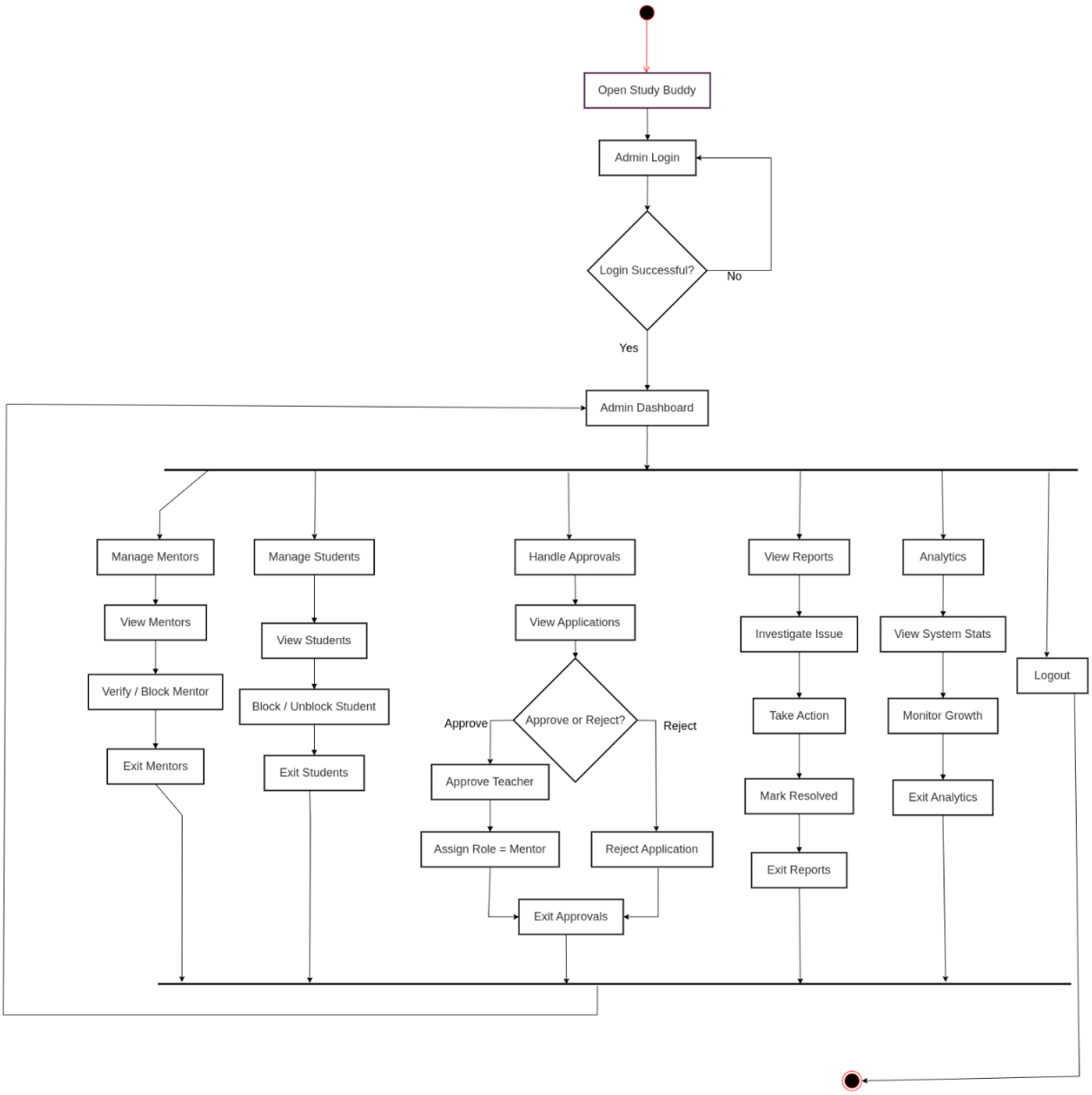


Figure 4.5 Process Flow/Admin Activity Diagram

4.4. Design Models

This section introduces the important design models that provides graphical representation of the StudyBuddy system structure and dynamic behaviour: Data flow diagram and System sequence diagram.

4.4.1. Data Flow Diagrams

We are following a sequential approach, so we have created Data flow diagrams till level 2 to explain our data flow and system in general.

I. Level 0 DFD

This diagram sets up the boundary of Study Buddy system by listing all the primary entities with which we interact. It validates that the main users who do the following input are Student and Mentor: Authentication, Booking Sessions, uploading resources, etc. Most importantly, it demonstrates that we need to depend on an outside system (Gemma3 AI Model) to service all our prompt requests and deliver the required generated notes/text. It shows that Study Buddy is the middle ground between the users and the powerful AI capabilities.

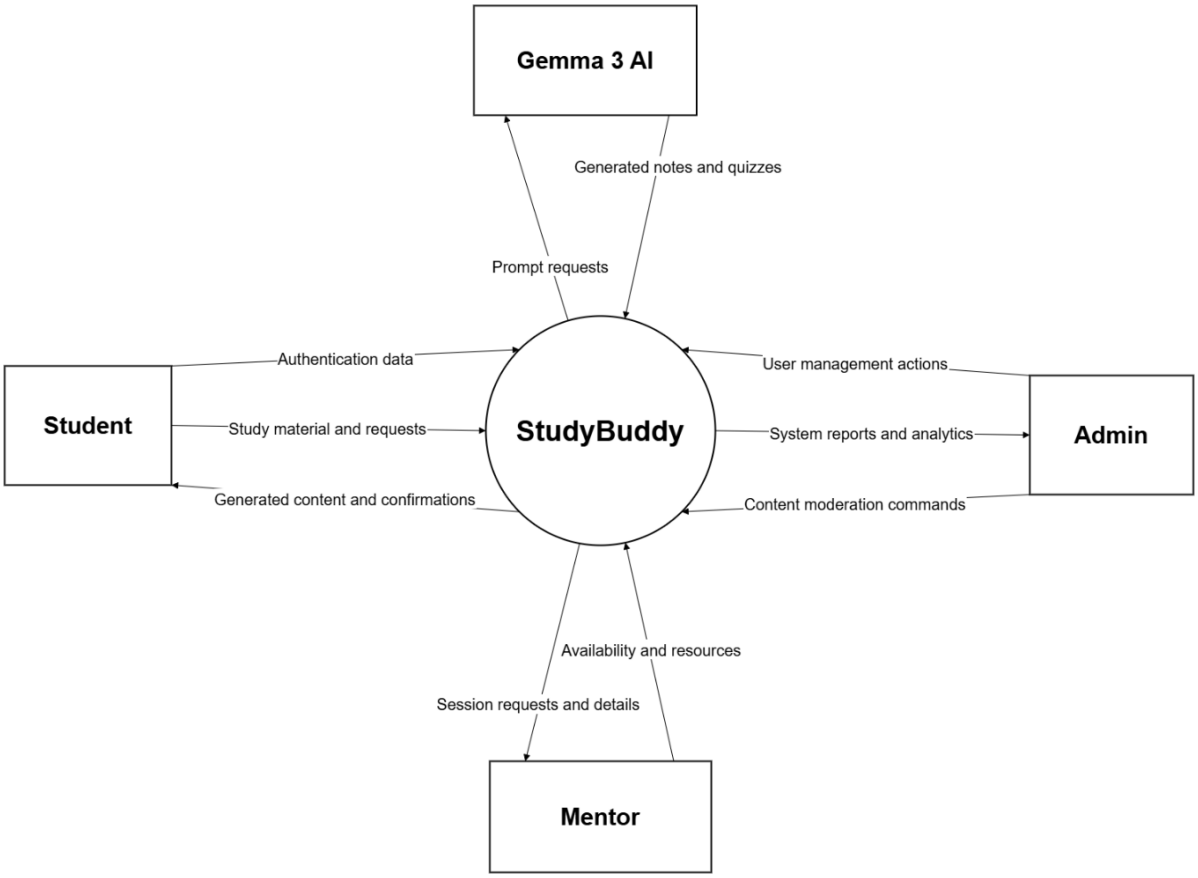


Figure 4.6 DFD Level 0

II. Level 1 DFD

The Level 1 DFD describes and depicts the inside of Study Buddy as nine fundamental operational modules. It illustrates the relationship of these processes (such as Mentorship and Content Generation) with our own databases (User DB, Notes DB), and how all system activities are controlled.

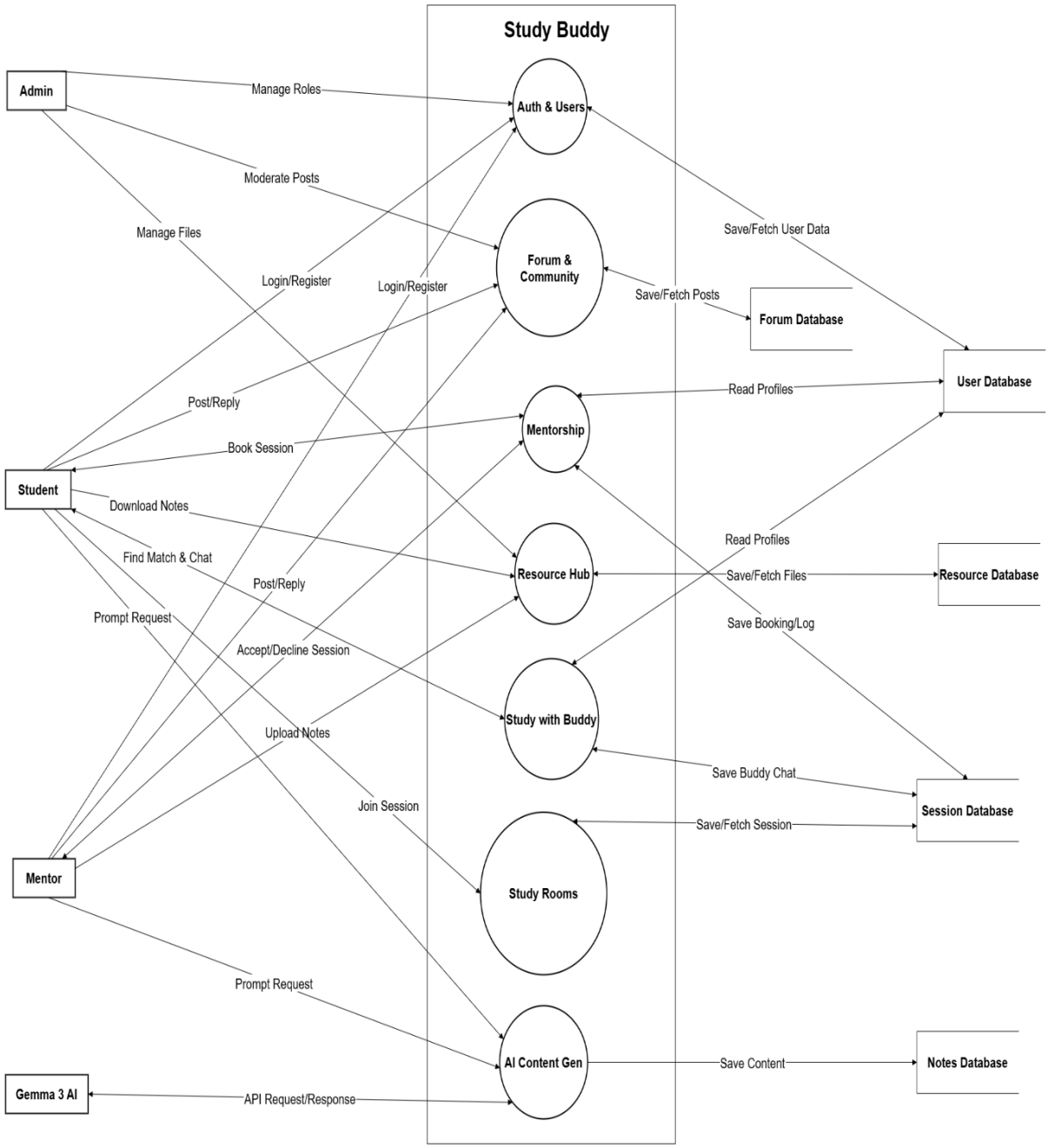


Figure 4.7 DFD Level 1

III. Level 2 DFD

The Level 2 Data Flow Diagrams allow for an examination of the internal operations of each of the larger modules of the Level 1 DFD. Level 2 displays diagrams for each of these critical functional processes (CFPs): Authentication & User Management, Content Generator, Mentorship etc.

i. Authentication and User Management

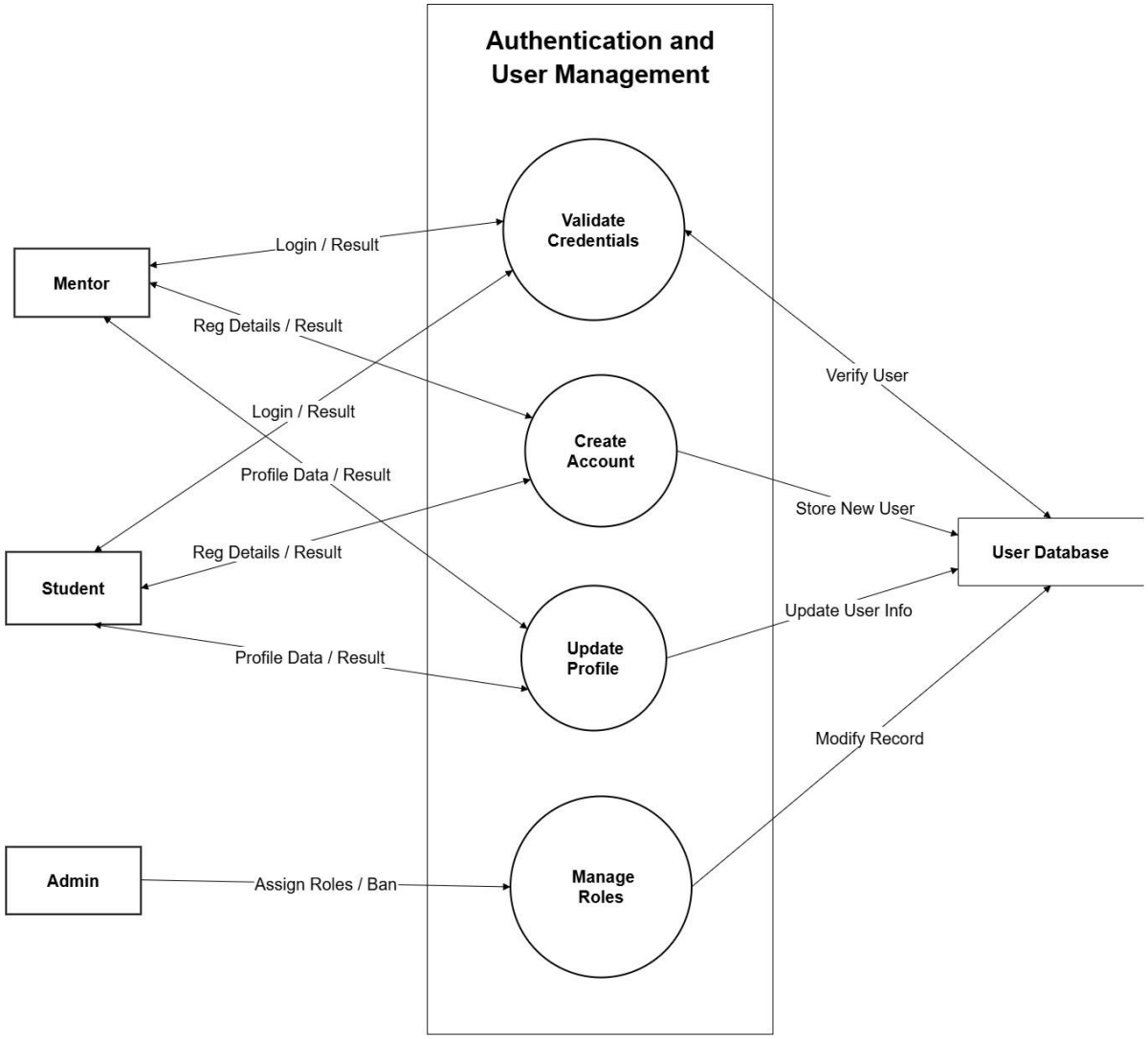


Figure 4.8 DFD Level 2(Authentication and User Management)

ii. Content Generator

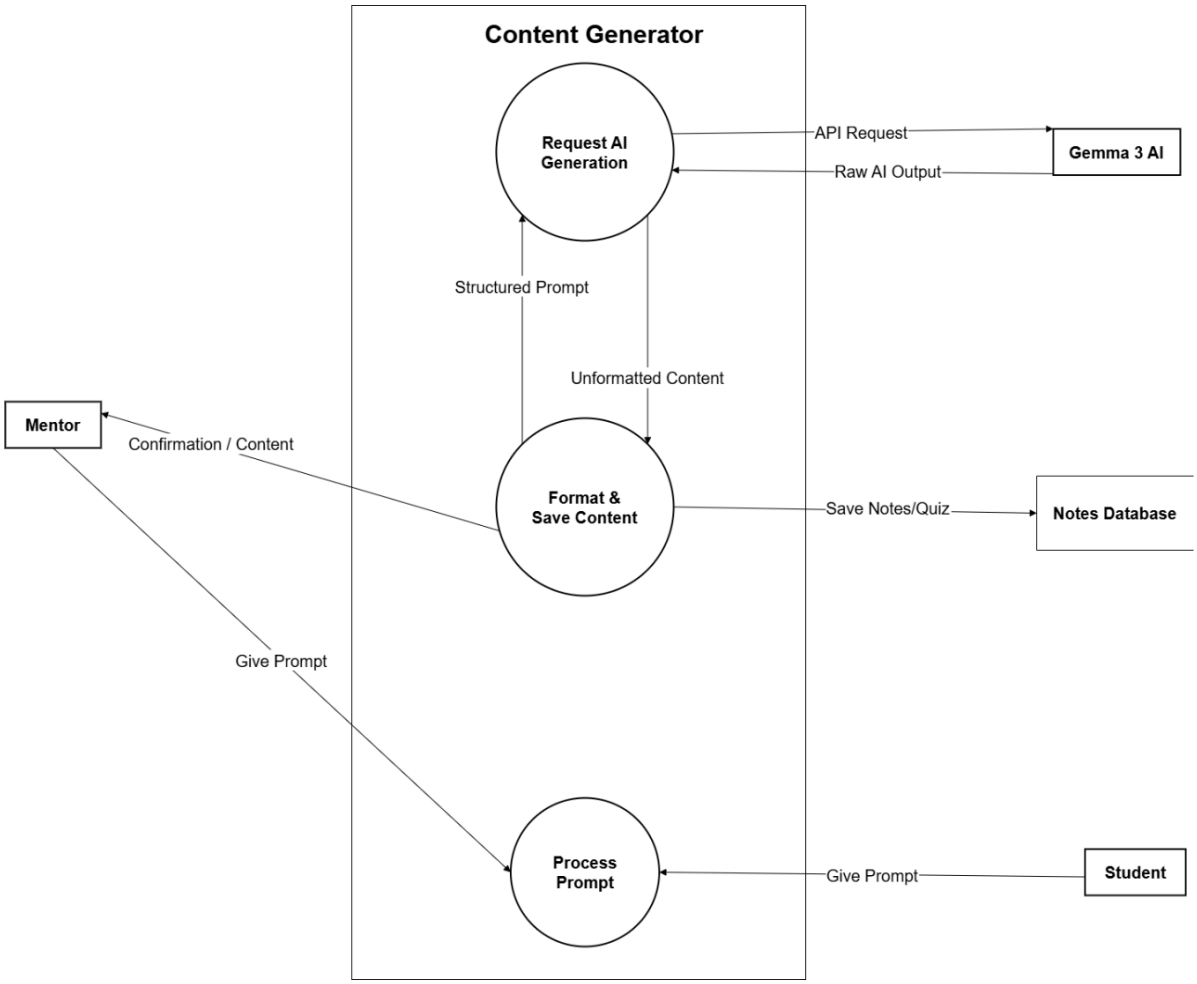


Figure 4.9 DFD Level 2(Content Generator)

iii. Study with Buddy

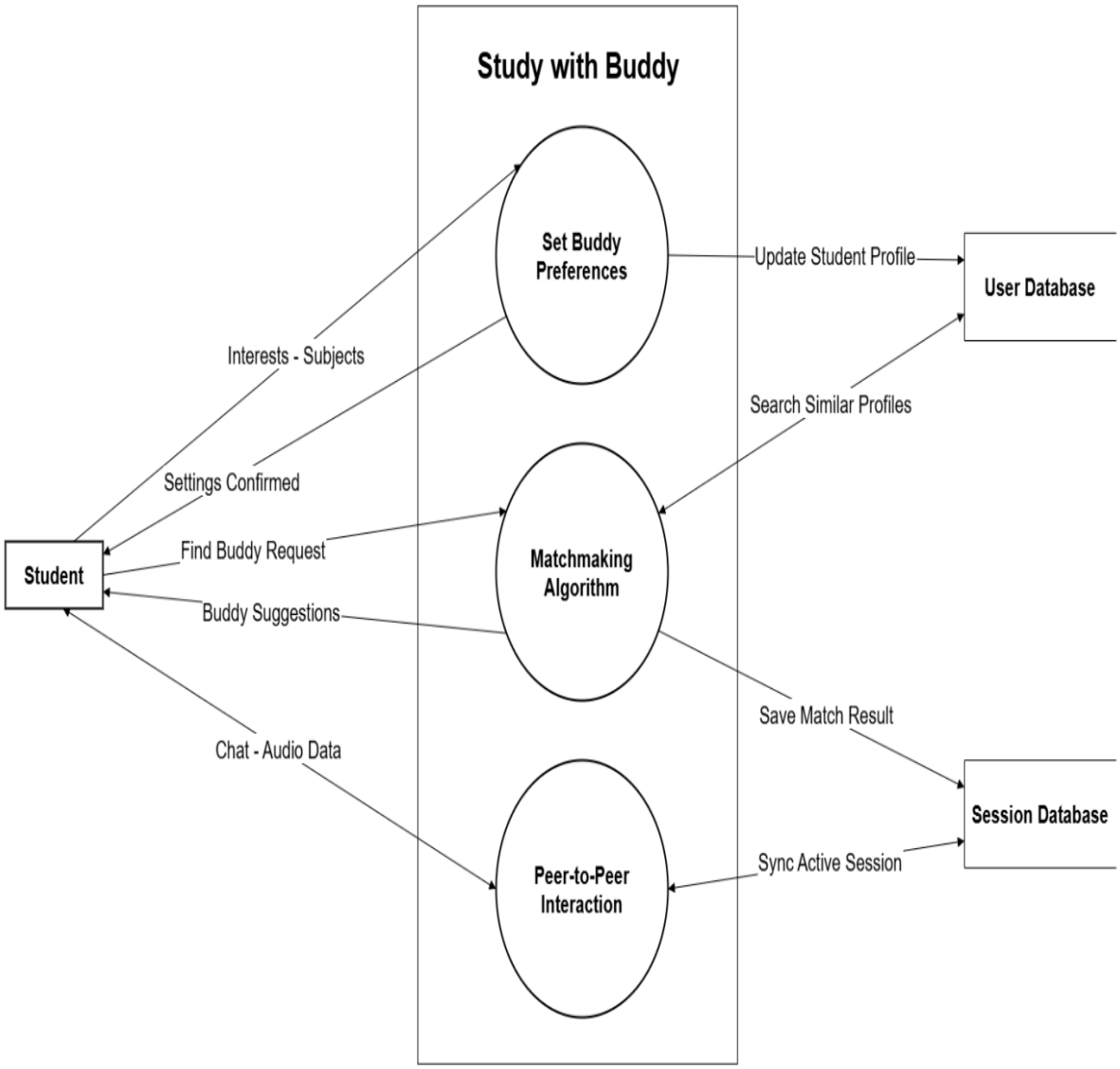


Figure 4.10 DFD Level 2(Study with Buddy)

IV. Study Room

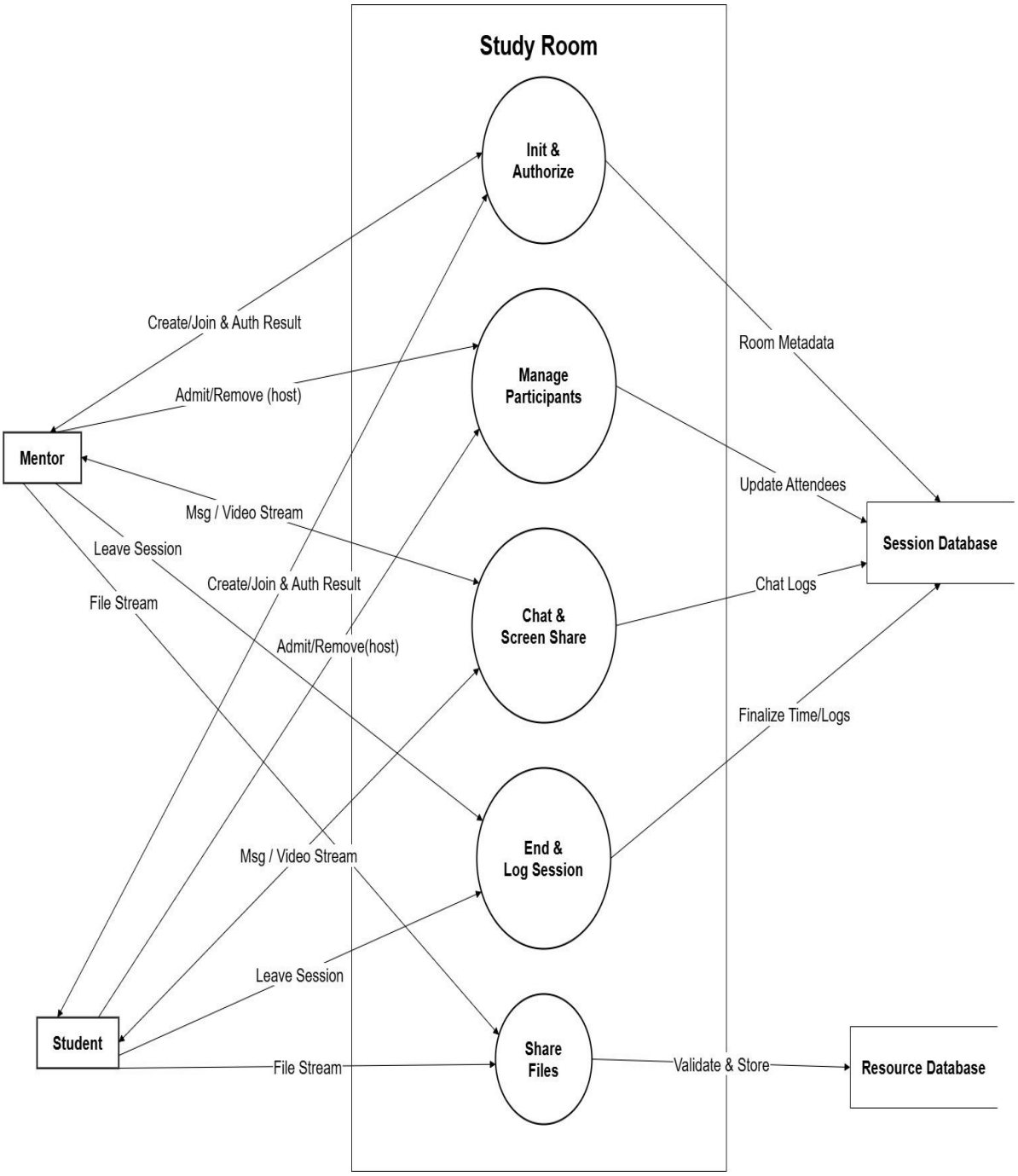


Figure 4.11 DFD Level 2(Study Room)

V. Community and Forums

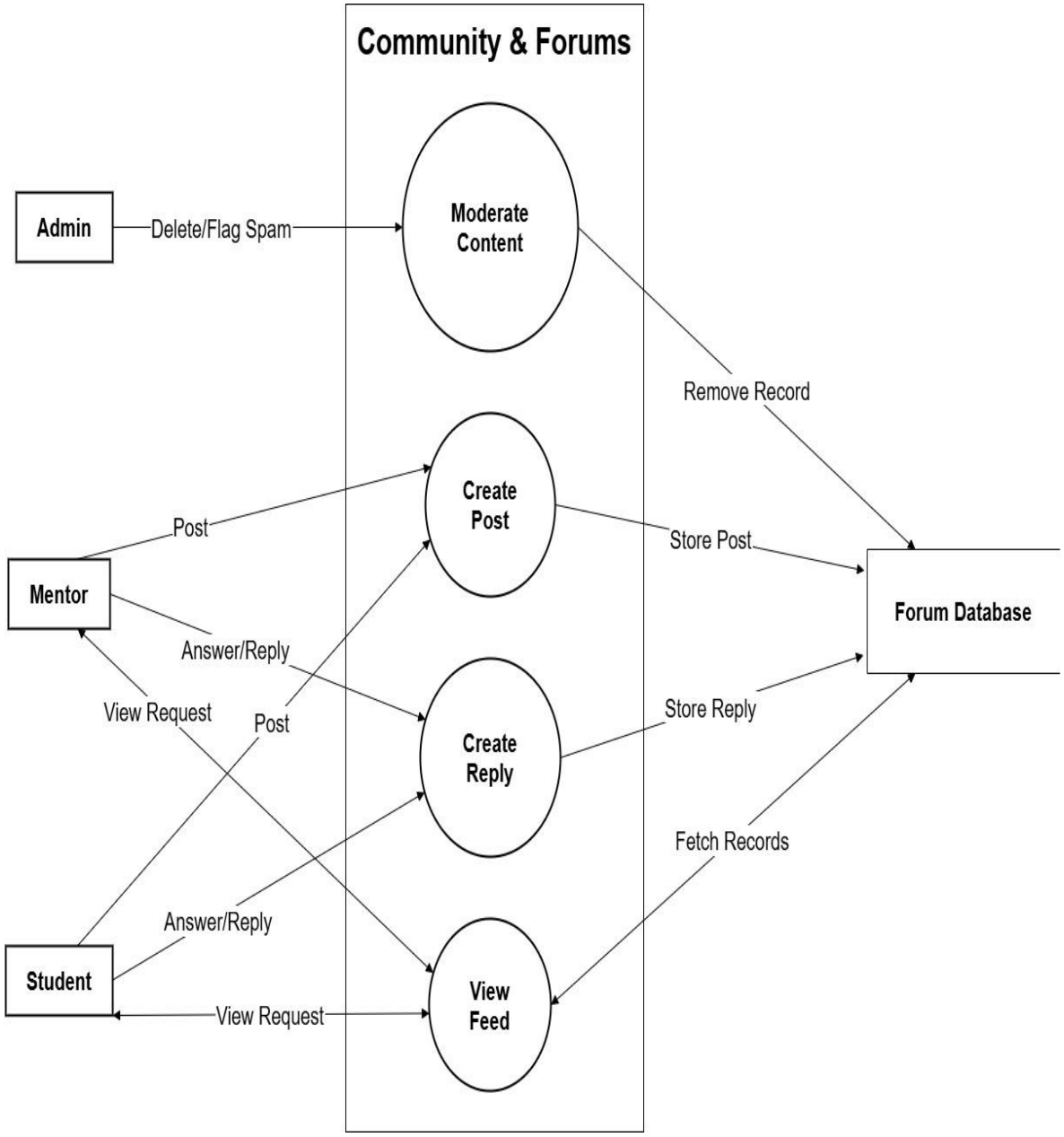


Figure 4.12 DFD Level 2(Community and Forums)

VI. Resource Hub

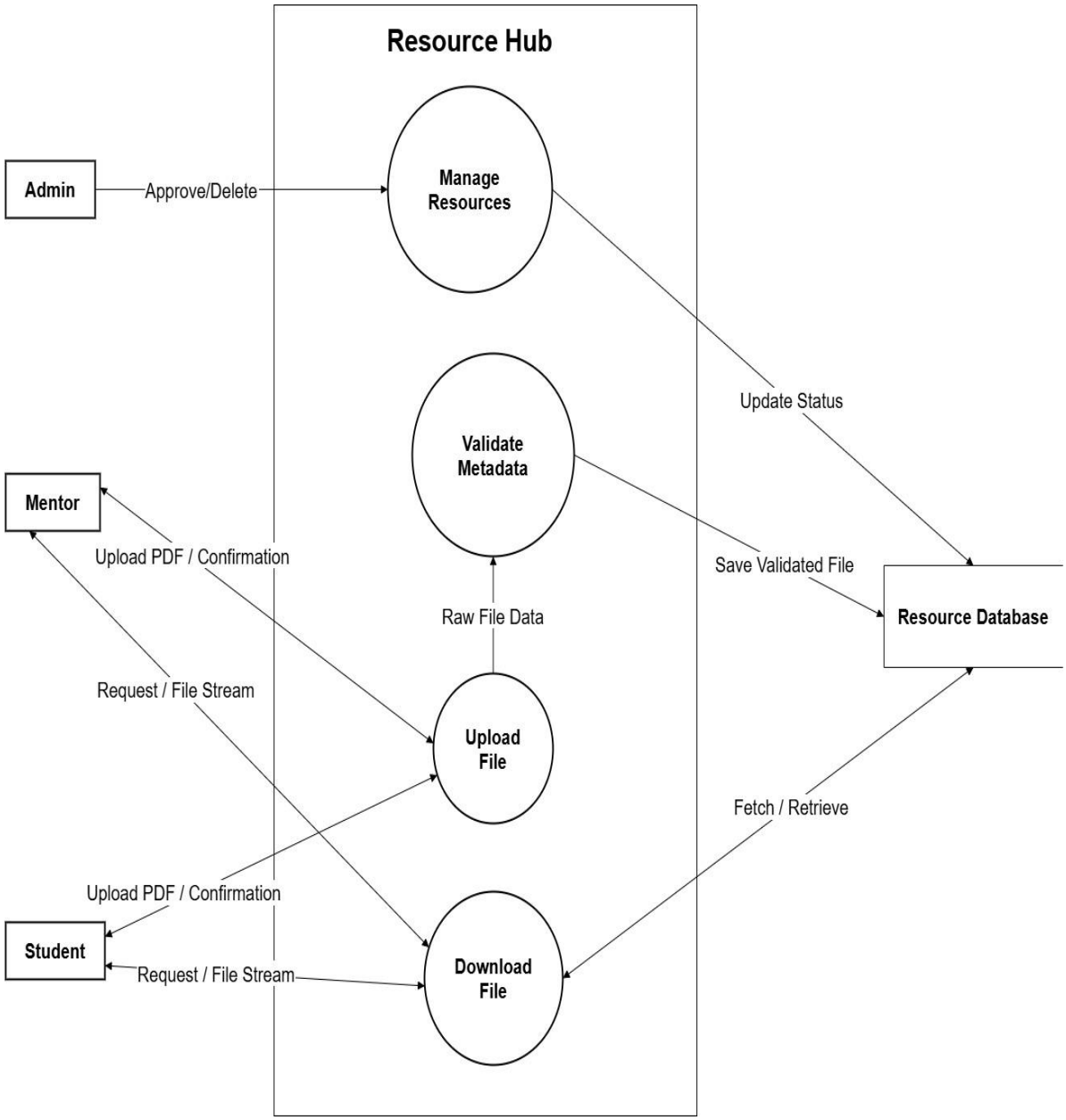


Figure 4.13 DFD Level 2(Resource Hub)

4.4.2. System Sequence Diagram

The following System Sequence Diagrams (SSDs) show the sequence of external messages and events that are sent between the Actor and the System. Each diagram describes a use case scenario, showing what the Actor does with the system, and what the system does in response, observable outputs.

i. Content Generator

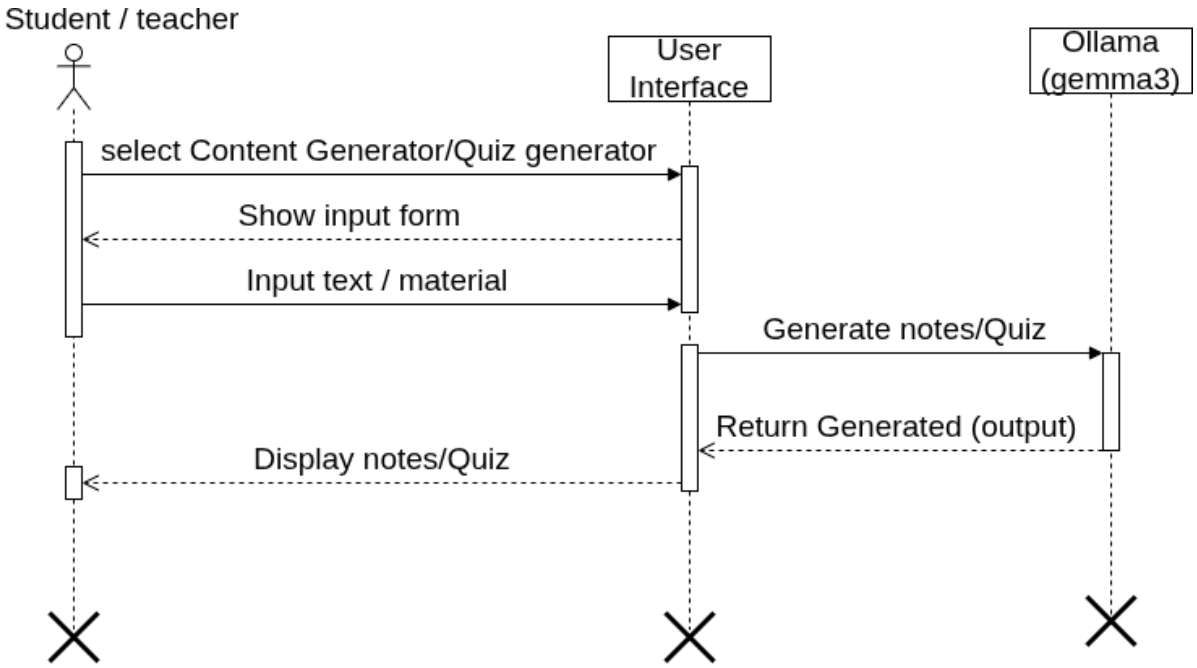


Figure 4.14 System Sequence Diagram (Content Generator)

ii. Study with Buddy

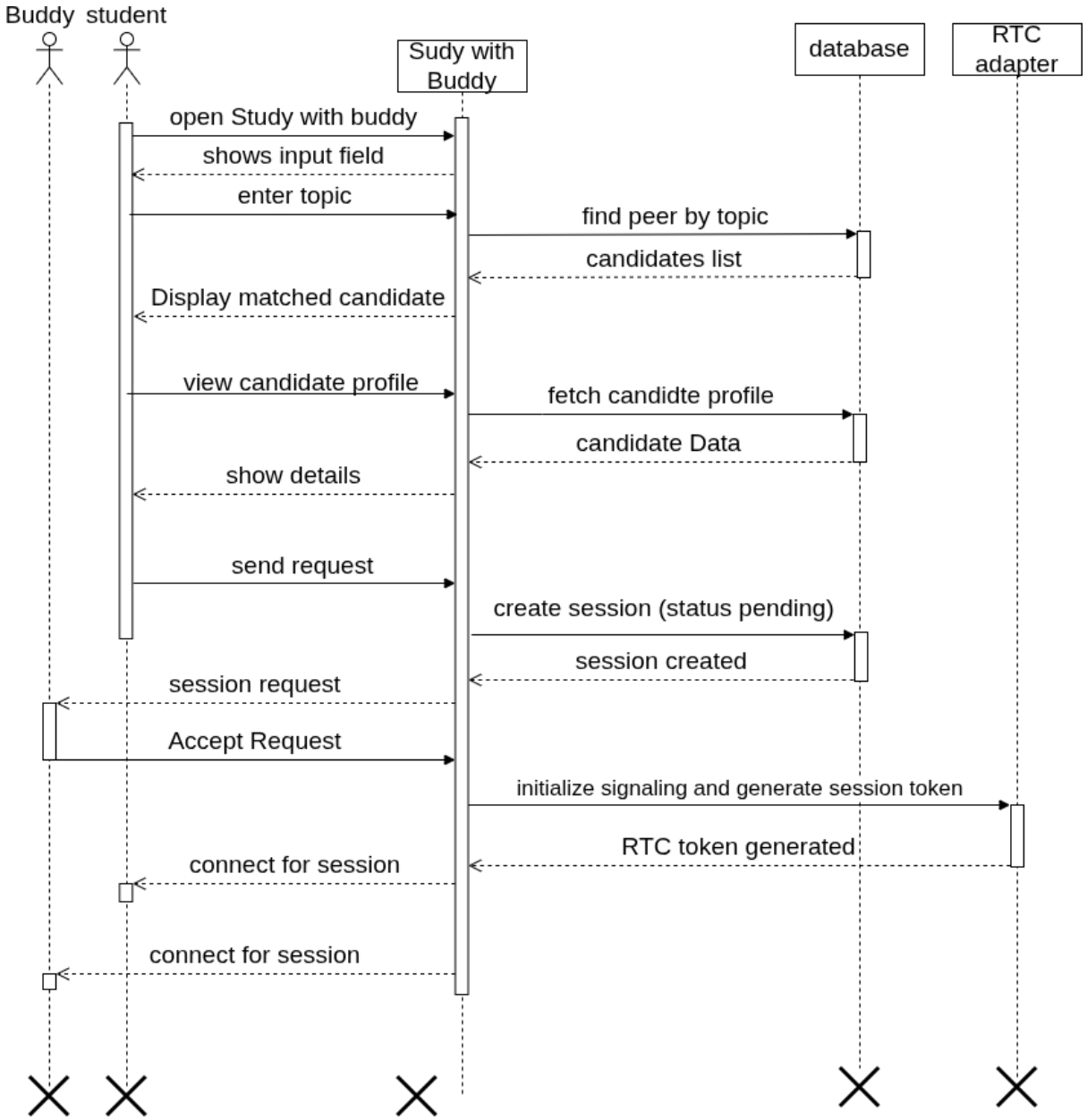


Figure 4.15 System Sequence Diagram (Study with Buddy)

iii. Mentorship

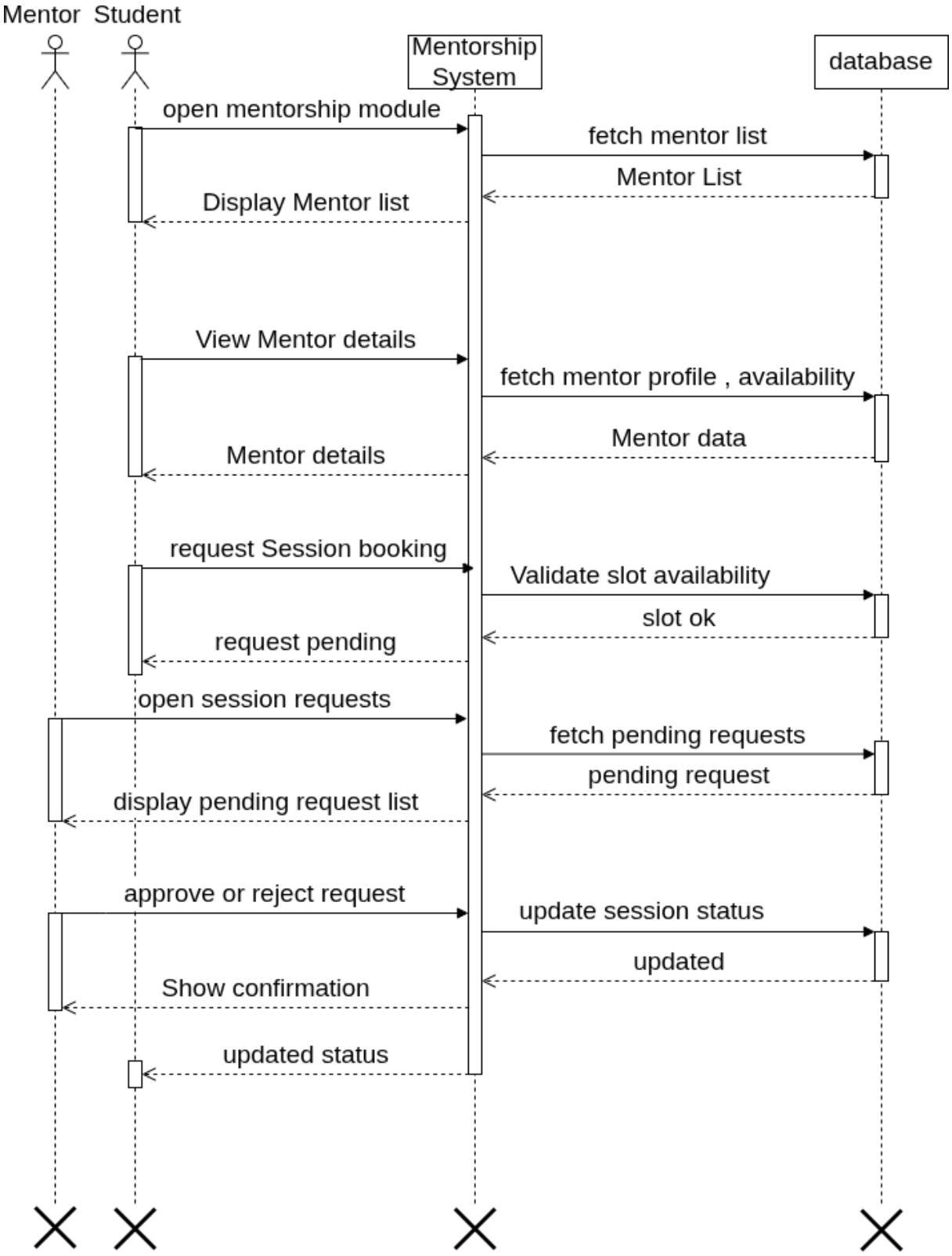


Figure 4.16 System Sequence Diagram (Mentorship)

iv. Study Room

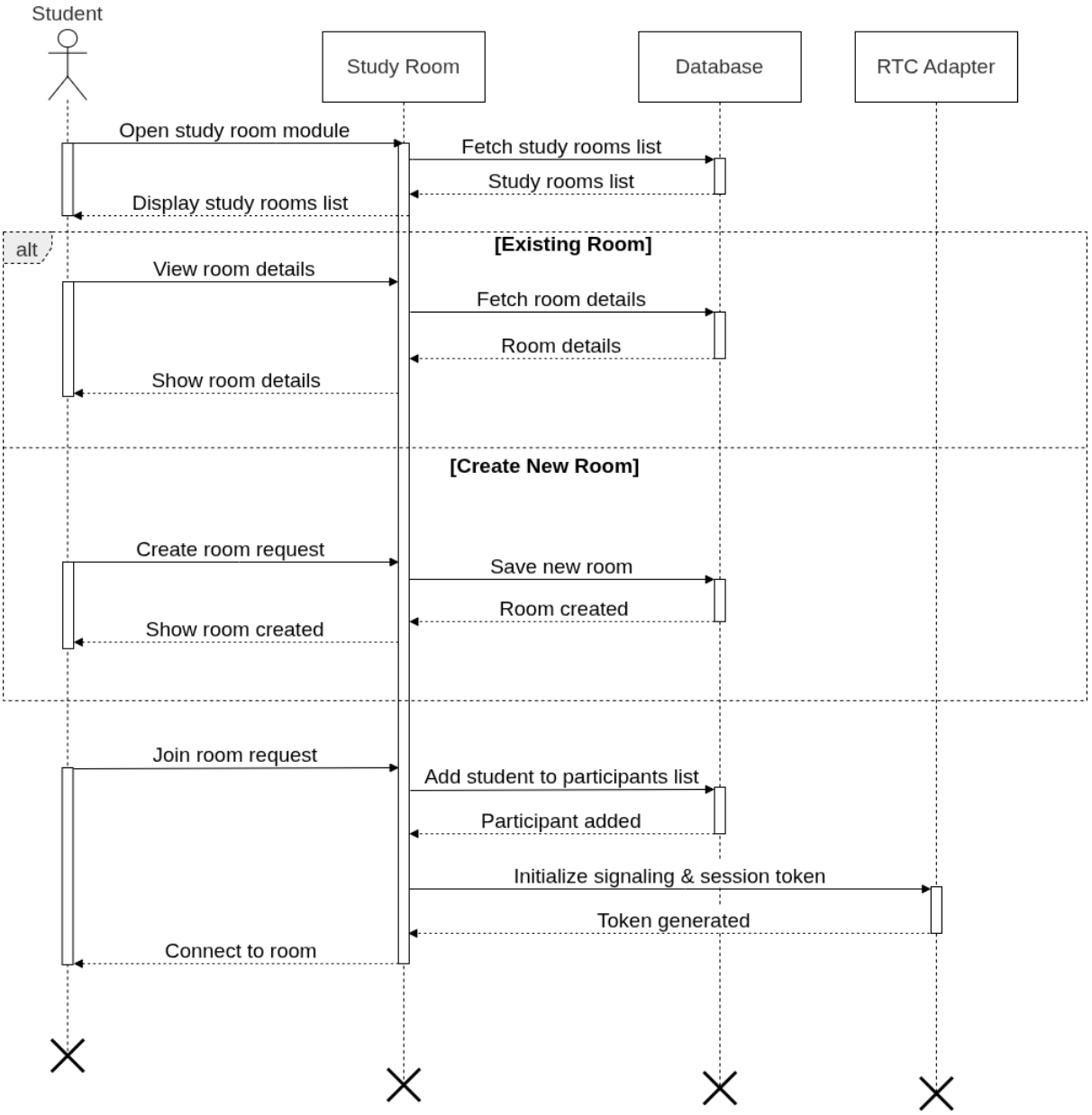


Figure 4.17 System Sequence Diagram (Study Room)

v. Resource Hub

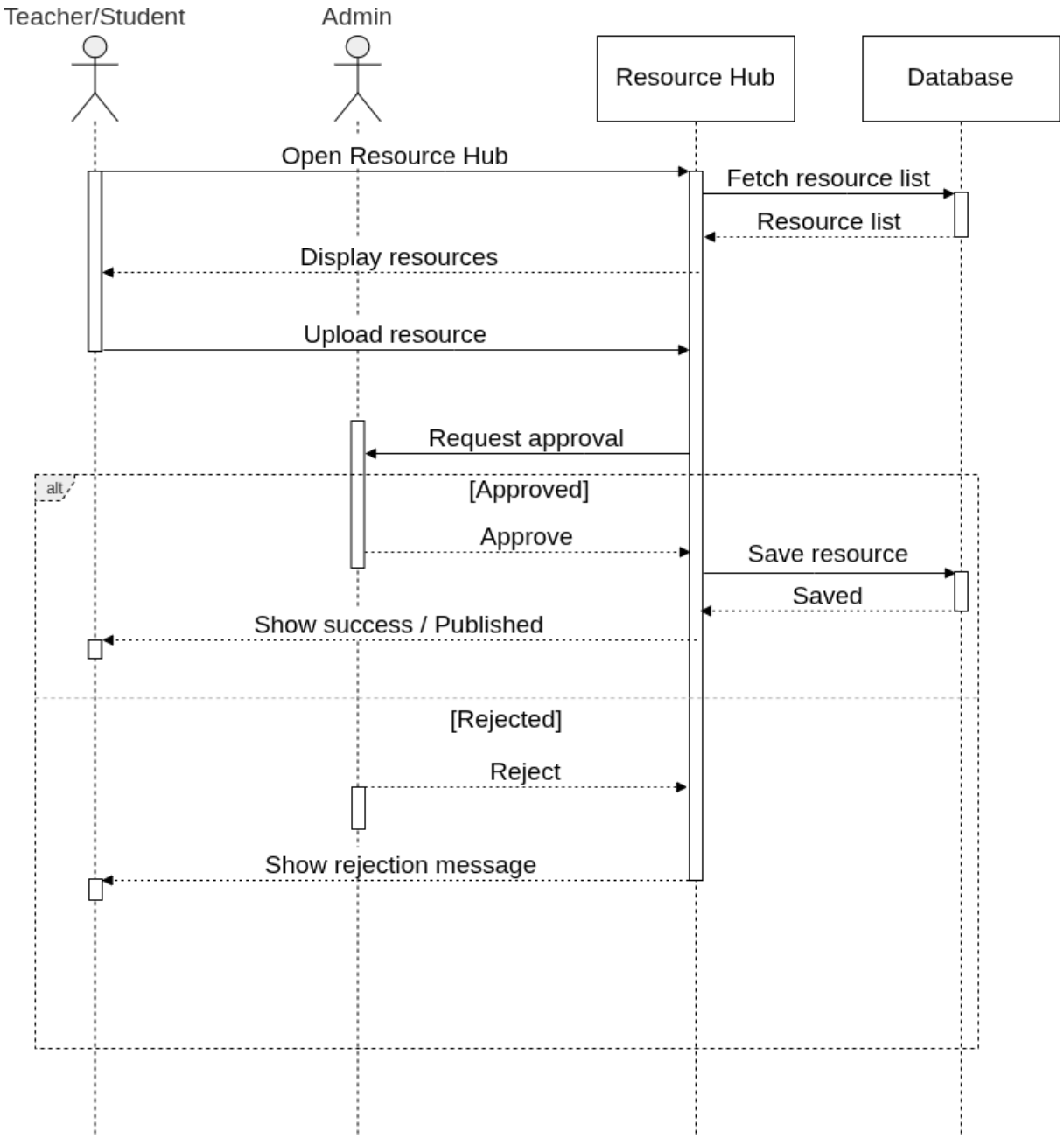


Figure 4.18 System Sequence Diagram (Resource Hub)

4.4.3. Package Diagram

The package diagram is a diagram that illustrates the modular structure of the StudyBuddy platform, organized into distinct functional layers. These include the Frontend Layer (React for user interface), the Backend Layer (Node.js and Express for business logic and API handling), the Database Layer (MongoDB collections for users, sessions, posts and resources), and the Real-Time Communication Module (WebRTC for live audio/video interaction). Plus, an AI Processing Package is included that will sync with an Ollama-hosted Gemma model, to process notes, summaries, and quizzes. An Admin and Moderation Package is a separate package from the core system that will be used to manage user verification, reports, and platform analytics. This separation of concerns allows for scalability, maintainability and ensures a clean architecture with real-time, AI and core application services all working separately but tightly together.

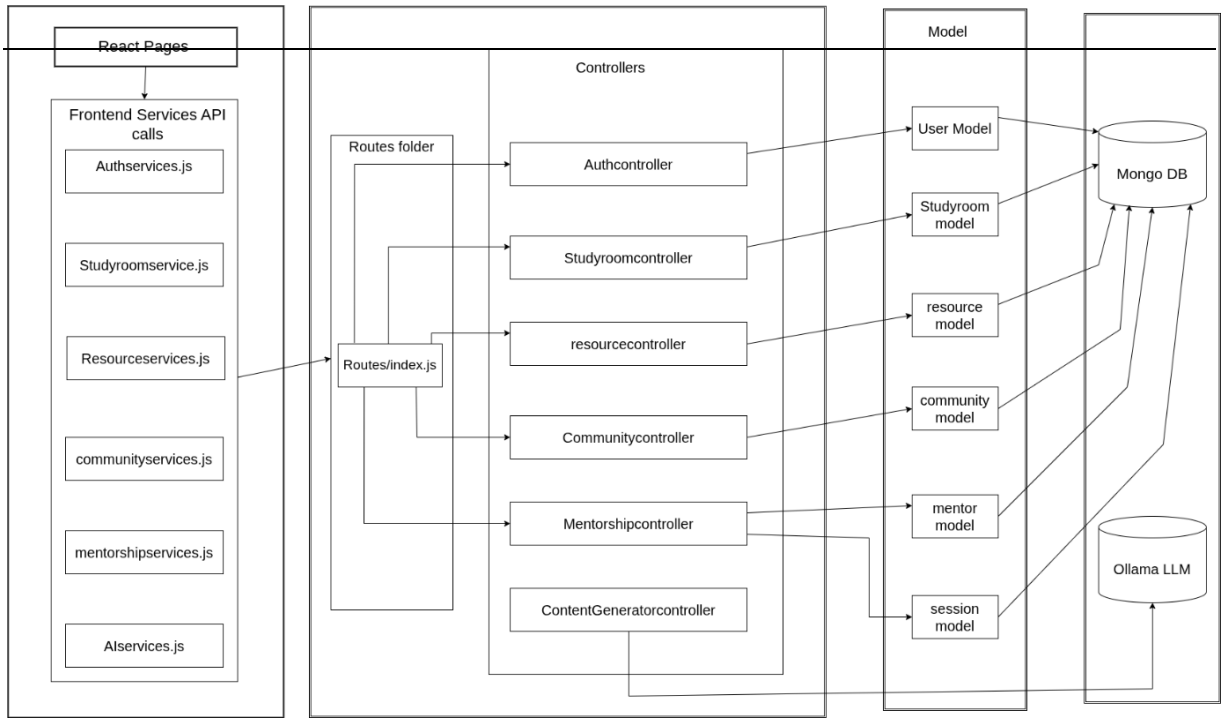


Figure 4.19 Package Diagram

5. Implementation

In this section we will summarize the functions that the StudyBuddy application should provide, and provide detailed information on the implementation of each of the modules. It begins with talking about the key algorithms that will be included in the system and what the algorithm is looking for and the step by step description of the algorithm. It then introduces the external APIs that are included into the app to power features such as artificial intelligence tool creation, real-time interaction, and secure user authentication. Last, the user interface (UI) design features the different screens that were created for the different user interactions, such as login, booking mentors, studying together, and creating content, to make it easy and intuitive for both students and instructors as well as for administrators.

5.1. Algorithm

The algorithm part represents logical steps and flow of the operations in the sections that are vital to numerous features of the StudyBuddy application to achieve efficient implementation of activities including: User authentication, Inactivation, Collaborative use of peers and Platform administration etc.

5.1.1. User Account Management

The following is a list of steps and objectives to administer user accounts.

Objective: Securely create and manage student and instructor accounts.

Input: User Registration/Login Request

Output: User Account Status / Authentication Response

Algorithm Steps

1. Validate all required user information, including name, email, password, and selected role.
2. Check if the inputted email address is registered or not.
3. Store the user's password in an encrypted bcrypt derived password.
4. Create the user profile and save account information in the database.
5. On login, check entered credentials with a list of stored credentials.
6. Generate a JWT access token after successful authentication.
7. Support optional Google OAuth 2.0 authentication for users selecting Google Sign-In.
8. Return the authentication status and user session details.

5.1.2. Content Generator (AI-Based Notes, Summaries, and Quizzes)

The goal and algorithm steps of the content generator are described below.

Objective: The goal is to enable students and teachers to create study notes with the help of AI from a subject or even uploaded text.

Input: Topic / Uploaded Text, Content Type

Output: Generated Study Material

Algorithm Steps

1. Accept the topic or uploaded study material from the user.
2. Validate that the input is not empty.
3. Organize the user's input into a prompt appropriate to the type of output, notes, summary, or quiz requested.
4. Send the prompt to the locally-hosted Ollama service running the Gemma 3 model and wait for the response.
5. Receive the generated AI response.
6. Store the generated content in the user's history.
7. Display the generated study material to the user.

5.1.3. Study With Buddy (Peer Matching)

The objective and algorithm steps of peer matching are as follows.

Objective: Match peers online who have similar study interests.

Input: Subject / Topic

Output: Matching Peer List

Algorithm Steps

1. Let student input the topic or subject he/she wishes to learn.
2. Search through the currently online students
3. Filter to find those who have the desired topic.
4. Exclude the requesting student from the results.
5. Show student a subset of matching peers from which to select.
6. Make a request to the selected peer.
7. If the peer accepts the request, initialize a Study Room session.
8. Return the matching result and session status.

5.1.4. Study Room (Real-Time Collaboration)

Objective: The Study Room's objective is to offer a collaborative, real-time environment where matched students or mentorship participants can connect.

Input: Accepted Study Request

Output: Active Study Room

Algorithm Steps

1. Verify that both participants have accepted the study request.
2. Generate a unique Study Room ID.
3. Store the room information in the database.
4. Initialize Socket.io for real-time communication.

5. Establish a WebRTC connection for audio and video communication.
6. Allow participants to upload and share study materials.
7. Save session details, including participants, start time, and end time.
8. Return the active room details.

5.1.5. Mentorship System

The objective and algorithm steps of the mentorship system are as follows.

Objective: The goal of the mentorship system is to enable students to schedule sessions with approved instructors.

Input: Mentor ID, Selected Time Slot

Output: Booking Status

Algorithm Steps

1. Show a list of available Mentors based on their subject and availability.
2. Give the student an opportunity to pick an open time slot and make a session request to the teacher.
3. Create a mentorship booking request.
4. Notify the selected instructor.
5. Update the booking status based on the instructor's response.
6. Reserve the selected time slot if accepted.
7. Return the final booking status.

5.1.6. Focus Room (Pomodoro Timer)

The objective and algorithm steps of the Focus Room are as follows.

Objective: To assist students in keeping the focus and consistent focus throughout independent study sessions.

Input: Session Start Request

Output: Focus Session Status

Algorithm Steps

1. Initialize a 25-minute study timer.
2. Monitor the timer until completion.
3. Record the completed focus session.
4. Notify the user that the study session has ended.
5. Start the optional 5-minute break timer.
6. Allow the user to begin another Pomodoro cycle or end the session.
7. Update the user's focus statistics.

8. Return the session summary.

5.1.7. Community and Forums

The objective and algorithm steps of the community module are as follows.

Objective: To facilitate the exchange of knowledge and discussion of academic topics between students and instructors.

Input: Post / Comment Request

Output: Updated Community Feed

Algorithm Steps

1. Validate the submitted post or comment.
2. Store the post with its title, description, and subject tag.
3. Retrieve community posts in chronological order.
4. Allow users to comment on existing posts.
5. Update the comment count after each new comment.
6. Record likes and engagement metrics.
7. Refresh the community feed.
8. Return the updated community posts.

5.1.8. Resource Hub

The Algorithm steps and the objective of the resource hub is as follows.

Objective: To control education resources shared by users.

Input: Resource File and Metadata

Output: Resource Status

Algorithm Steps

1. Check uploaded resource file.
2. Upload the file to cloud storage.
3. Save the file metadata and storage URL in the database.
4. Allow users to search resources by subject or popularity.
5. Retrieve the requested resource.
6. Update the download count after each download.
7. Display the resource information.
8. Return the resource status.

5.1.9. Admin Controls

The objective and algorithm steps of admin controls are as follows.

Objective: Monitor users and maintain platform integrity.

Input: Administrative Request

Output: Updated Platform Status

Algorithm Steps

1. Retrieve pending instructor verification requests.
2. Approve or reject mentor applications.
3. View registered students and instructors.
4. Block or unblock user accounts when necessary.
5. Review reported posts and uploaded resources.
6. Remove content violating platform policies.
7. Generate platform analytics, including users, active sessions, and uploaded resources.
8. Return the updated administrative status.

5.2. External APIs

The following external APIs are used in our project. Table 5.2 shows the details of APIs used in the project.

Table 5.1: Details of APIs used in the project

Name of API	Description of API	Purpose of usage	function in which it is used
Ollama API (Gemma 3)	Local large language model server exposing REST APIs for AI text generation using the Gemma 3 model.	Generates AI-powered notes, summaries, and quizzes from user-provided topics or uploaded study material.	generateAIContent() – contentController.js
Google Identity Services API (OAuth 2.0)	Authentication platform that verifies Google user identities and returns authenticated profile information.	Enables secure Google Sign-In for students and instructors during registration and login..	loginWithGoogle() – authController.js
Socket.io	Real-time bidirectional communication library built on WebSocket technology.	Handles WebRTC signalling and enables real-time messaging during study and mentorship sessions.	handleSignal() – studyRoomService.js

Name of API	Description of API	Purpose of usage	function in which it is used
WebRTC	Browser-based peer-to-peer communication standard for real-time audio, video, and data exchange.	Establishes live audio and video communication between study partners and mentors.	StudyRoom.jsx, MentorshipSession.jsx
Cloud Object Storage API (S3-compatible)	REST-based object storage service for uploading and retrieving files through unique URLs.	Stores study resources and uploaded files while maintaining file metadata in MongoDB.	uploadResource() – resourceController.js

5.3. User Interface

The StudyBuddy application's user interface (UI) has been designed in a simple manner, usability, and accessibility in mind. The design will ensure that students, educators and visitors will have a positive learning experience, administrators stay up to date with the platform and interact easily with it, following the guidelines, aesthetic considerations and the functional needs outlined in Chapter 3.

5.3.1. Login Screen

Allows students and instructors to log into the screen securely with their e-mail and password, Or with a single log-in to Google. A "Login as Teacher" toggle switches the destination There's a link for folks who don't have a dashboard yet, and no changes to the form fields an account.

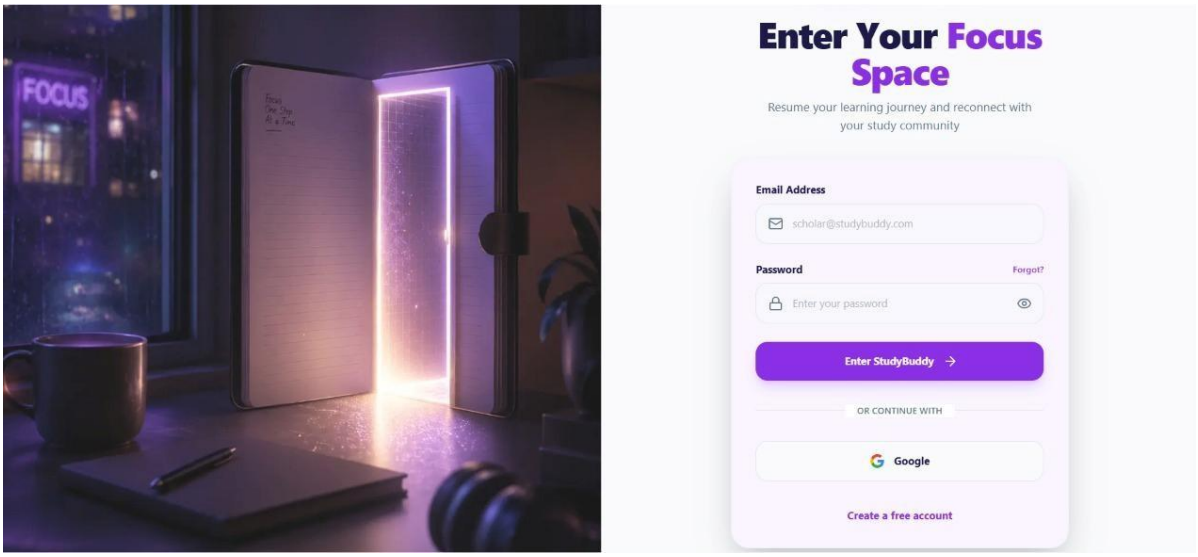


Figure 5.1 StuddyBuddy Login Screen

5.3.2. Registration Screen

New users will be able to register using this screen by entering the new user's full name, email, password and role. It offers safe login – password is hashed before login stored, and Real-time validation of required fields.

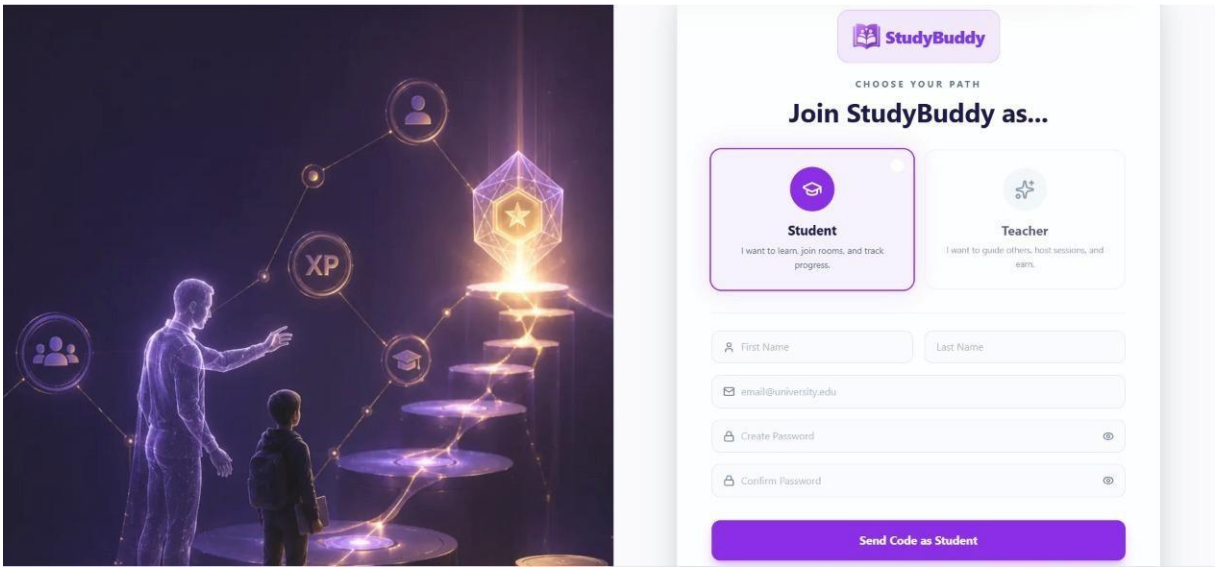


Figure 5.2 StuddyBuddy Registration Screen

5.3.3. Student Dashboard Screen

The Student Dashboard gives a customised view of the progress the student has made and this consists of: study streak, total study time, number of active study matches and level progress. It is also possible to appear recent Study With Buddy and mentoring icons of access, and activity reservations.

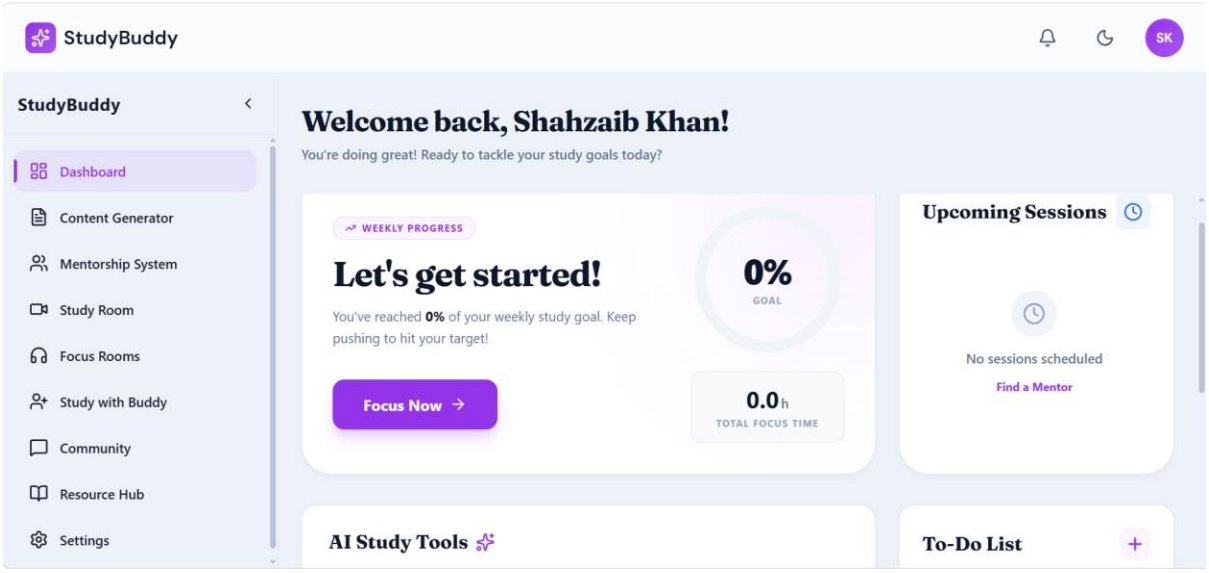


Figure 5.3 StuddyBuddy Student Dashboard

5.3.4. Instructor Dashboard Screen

The Instructor Dashboard provides an overview of students, upcoming sessions, and provides a history of students' performance. Requests that are pending, and resources uploaded. It includes forthcoming sessions with join controls, pending session requests with accept/deny actions, a summary of each student's progress.

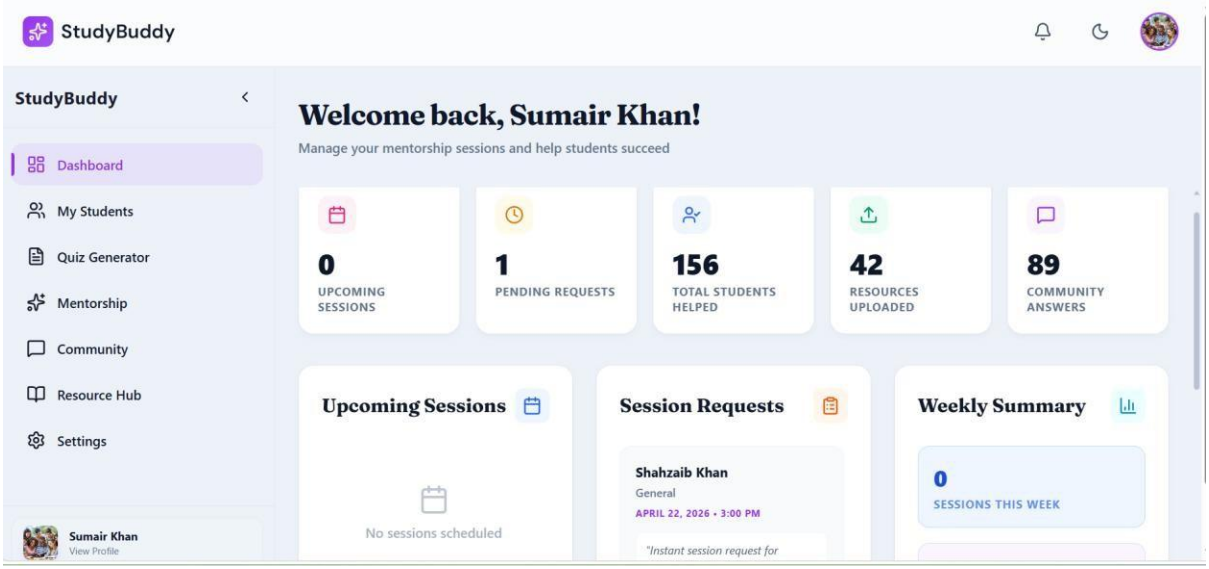


Figure 5.4 StuddyBuddy Instructor Dashboard

5.3.5. Content Generator Screen

This screen displays a dialogue box that the student and teacher can use to type a topic or Copy and paste study content, the AI tool generates structured notes, a summary, or a quiz. If the mode is selected, it depends.

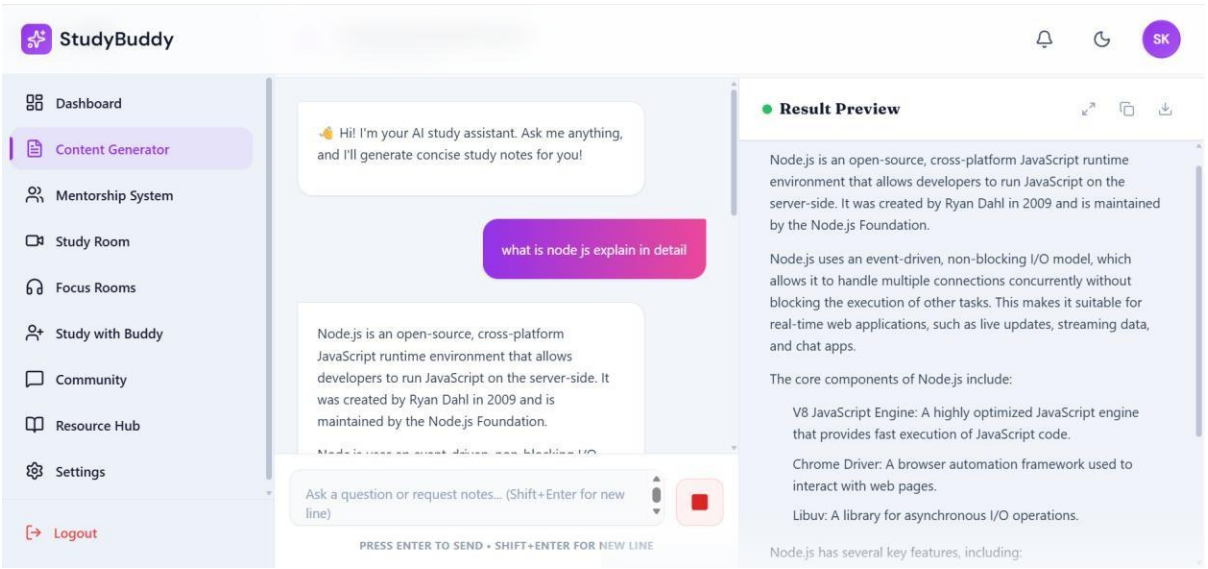


Figure 5.5 StuddyBuddy Content Generator Screen

5.3.6. Focus Room Screen

The Focus Room features a clean, clutter-free design with a central Pomodoro timer, allowing the student to start a work session and respond to break prompts once a cycle completes.

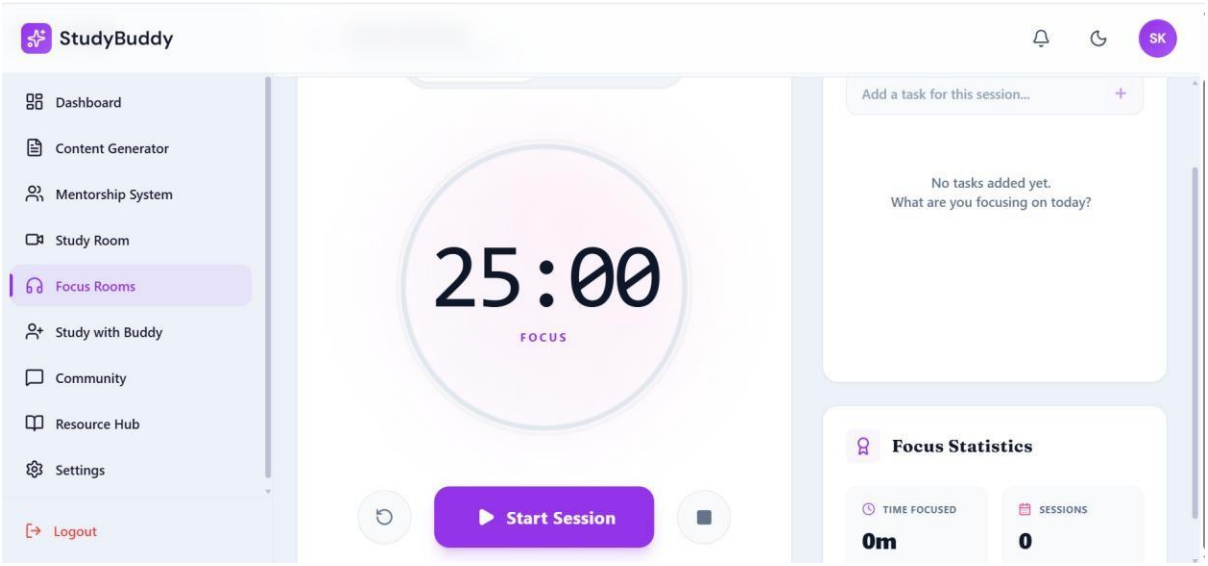


Figure 5.6 StuddyBuddy Focus Room Screen

5.3.7. Study With Buddy Screen

This screen will let a student type a subject and see a list of students that are currently online who are sharing that subject which shows a candidate's profile and allows him/her to send a study request.

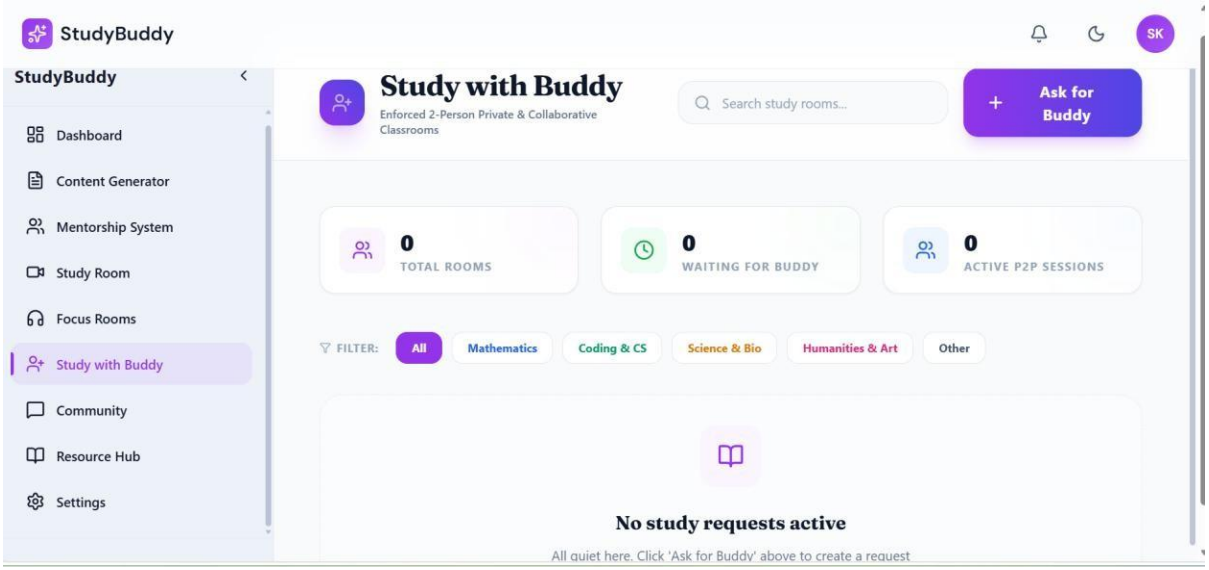


Figure 5.7 StuddyBuddy Study with Buddy Screen

5.3.8. Mentorship Screen

This screen allows students to browse available mentors filtered by subject, view a mentor's availability, and submit a session booking request for an open time slot.

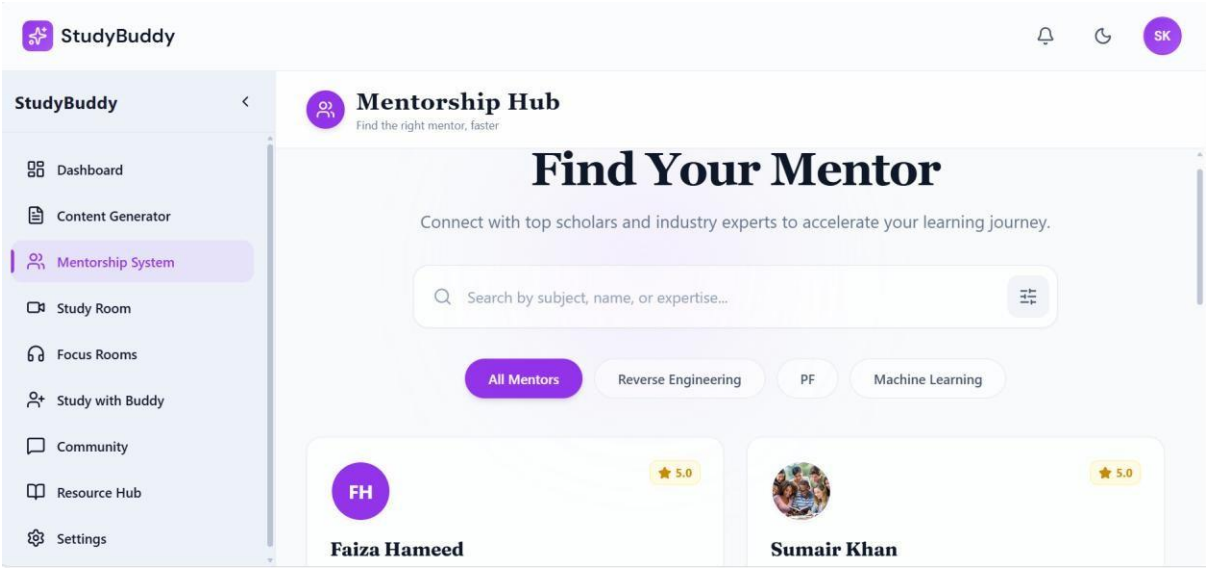


Figure 5.8 StuddyBuddy Mentorship System Screen

5.3.9. Study Room Screen

The Study Room screen combines a live audio and video panel, real-time text chat, and a shared materials list, allowing participants to communicate and exchange files during a collaborative session.

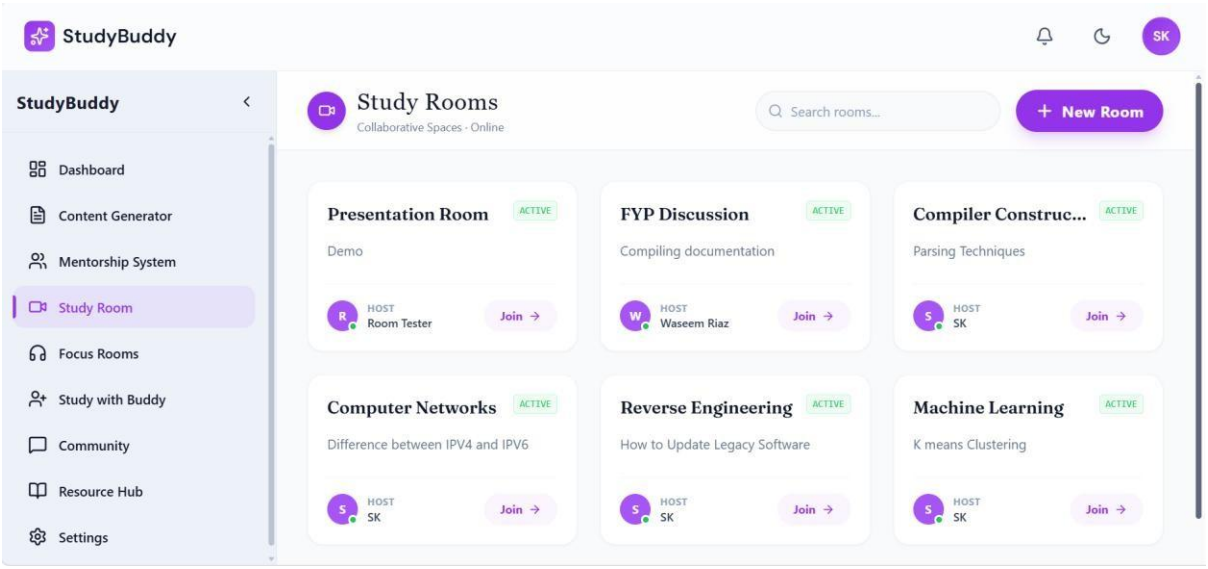


Figure 5.9 StuddyBuddy Study Room Screen

5.3.10. Community Screen

This screen displays a feed of posts created by students and instructors, with options to create a new post, browse existing discussions, and add comments.

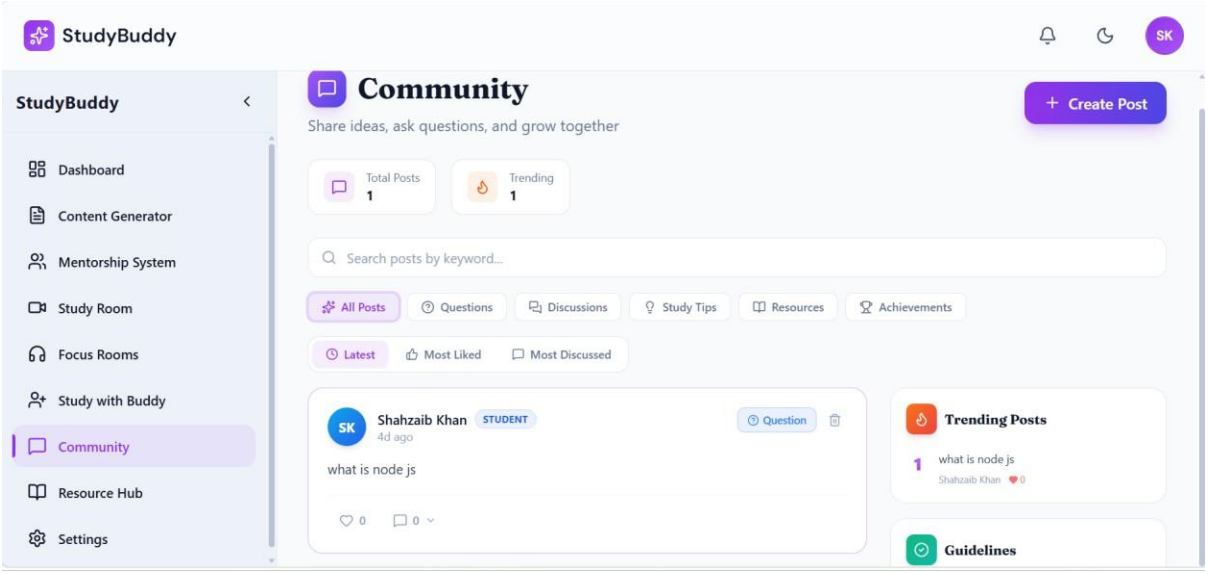


Figure 5.10 StuddyBuddy Community Screen

5.3.11. Resource Hub Screen

This screen shows a community searchable marketplace of study materials that are uploaded by the community, displaying subject and rating for each resource and uploading a new resource or download an existing one.

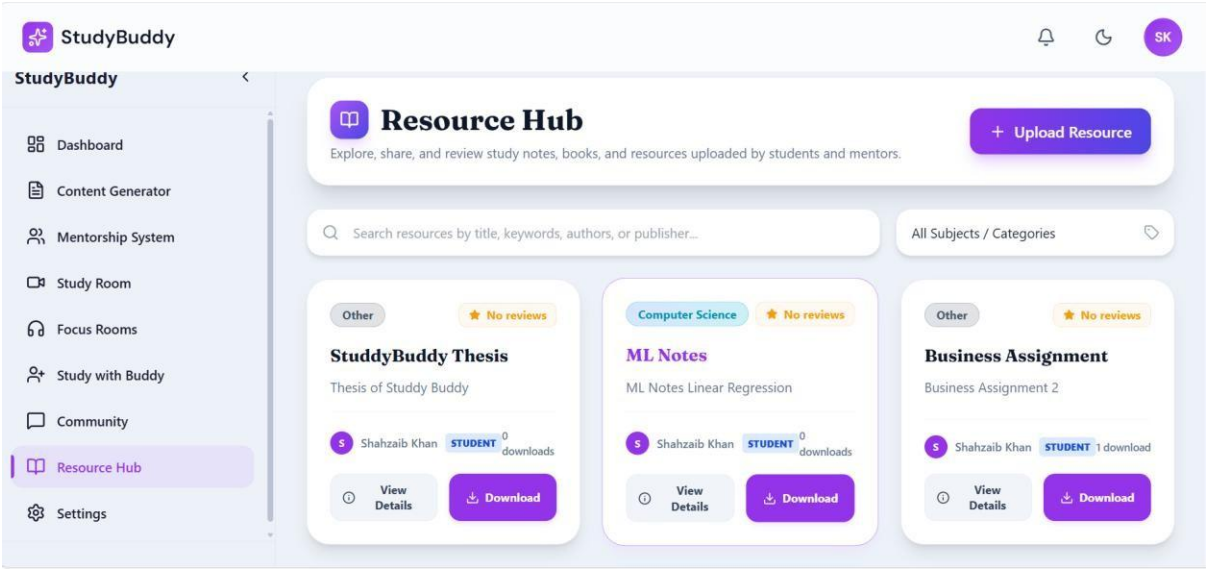


Figure 5.11 StuddyBuddy Resource Hub Screen

6. Testing and Evaluation

The Testing Phase is an essential part of the Software Development Life Cycle. For StudyBuddy, to ensure each module works correctly in, a multi-layered testing approach was taken. The ability to isolate and that the integrated system fulfills all functional and non-functional requirements defined in the SRS. A test was carried out at four levels: System Testing, Unit Testing, Functional Testing, Integration Testing, and Automated Testing.

6.1. Manual Testing

The development team conducted manual testing to ensure that the user-facing aspects of the system are accurate. Functionality, edge case behaviour, and overall system responsiveness without using any automated tools.

6.1.1. System Testing

Tests the entire integrated StudyBuddy application to ensure that it works as intended, all components operate together as expected. The following table shows the system test

results

Table 6.1: System Testing

Test Case	Description	Expected Result	Actual Result	Status
User Registration	Verify that a new student or instructor can register with valid details.	Account created and user redirected to login.	Registration successful.	Pass
User Login	Test authentication with valid JWT credentials.	User authenticated and redirected to dashboard.	Login successful.	Pass
AI Note Generation	Student enters a topic; system generates notes via Gemma3.	Relevant notes returned within 1 minute.	Notes generated correctly.	Pass
AI Quiz Generation	Instructor enters topic; system generates a quiz.	Quiz with questions displayed correctly.	Quiz generated and saved.	Pass
Focus Room Timer	Student starts Pomodoro timer; 25-minute work and 5-minute break cycles run.	Timer runs and alerts user at cycle end.	Timer functioned correctly.	Pass

Test Case	Description	Expected Result	Actual Result	Status
Study With Buddy Matching	Student searches for a peer by subject algorithm returns matches.	Matched peers displayed.	Relevant matches returned.	Pass
Peer Session (WebRTC)	Two matched students connect for a video study session.	Session established with audio and video.	Session connected successfully.	Pass
Mentorship Booking	Student books a session with an available mentor.	Request created with status 'Pending'.	Booking request saved.	Pass
Session Accept/Reject	Instructor accepts or rejects a pending session request.	Status updated; student notified.	Status updated correctly.	Pass
Community Post	User creates a new post in the community forum.	Post published and visible in forum feed.	Post saved and displayed.	Pass
Resource Upload	User uploads a study resource (PDF/notes) to Resource Hub.	File stored and listed in hub.	Upload successful.	Pass
Resource Download	User downloads a resource from the hub.	File transfer initiated on device.	Download initiated.	Pass
Study Room Join	Student joins an existing collaborative study room.	Room interface loads; participant added.	Room joined successfully.	Pass
Logout	User logs out; JWT invalidated.	Session ended; user redirected to login page.	Logout successful.	Pass

6.1.2. Unit Testing

There are unit tests for individual functions and components of the StudyBuddy system in isolation. The following tables contain unit test cases for most important modules.

- **Unit Testing 1:** Website Registration page as shown in Table 6.2
  Testing Objective: To ensure the Registration form is working correctly

Table 6.2: User Registration Unit Test Cases

No.	Test Case	Input Values	Expected Result	Result
1	Register with valid data.	Name: Waseem, Email: waseem@gmail.com, Password: Pass@123, Role: Student	User registered and redirected to login page.	Pass
2	Register with an already existing email.	Email: waseem@gmail.com (duplicate)	System rejects and displays 'Email already registered' error.	Pass
3	Register with invalid email format.	Email: waseemgmail (no @)	System displays 'Invalid email format' message.	Pass
4	Register with blank password field.	Password: (empty)	System prompts 'Password is required'.	Pass

- **Unit Testing 2:** Website Login page as shown in Table 6.3
  Testing Objective: To ensure the login form is working correctly

Table 6.3: User Login Unit Test Cases

No.	Test Case	Input Values	Expected Result	Result
1	Login with correct credentials.	Email: waseem@gmail.com, Password: Pass@123	JWT generated; user redirected to dashboard.	Pass
2	Login with incorrect password.	Email: waseem@gmail.com, Password: wrong123	System displays 'Invalid credentials' error.	Pass
3	Login with unregistered email.	Email: unknown@abc.com, Password: test123	System displays 'User not found' message.	Pass

No.	Test Case	Input Values	Expected Result	Result
4	Login with blank email field.	Email: (empty), Password: Pass@123	System prompts 'Email is required'.	Pass
5	Login with blank password field.	Email: waseem@gmail.com, Password: (empty)	System prompts 'Password is required'.	Pass

- **Unit Testing 3:** Content Generator page as shown in Table 6.4
  Testing Objective: To ensure the AI Content Generator is working correctly

Table 6.4: AI Content Generator Unit Test Cases

No.	Test Case	Input Values	Expected Result	Result
1	Generate notes with a valid topic.	Prompt: 'Explain Object Oriented Programming'	Structured notes returned within 15 seconds.	Pass
2	Generate a quiz with a valid topic.	Topic: 'React Hooks', Questions: 5	5 MCQ questions generated and displayed.	Pass
3	Submit empty prompt to content generator.	Prompt: (empty)	System displays 'Please enter a topic or prompt'.	Pass
4	Test when AI service is unavailable.	Ollama service offline	System displays 'AI Service Unavailable. Please try again later'.	Pass

6.1.3. Functional Testing

Functional testing to validate each feature of StudyBuddy works as expected. The products are made according to the functional requirements stated in the SRS. The most important are listed below.

- Functional test cases in all modules.

Table 6.5: Functional Testing

No	Test Case	Input / Action	Expected Result	Result
1	Register a new student account.	Name: Hamza, Email: hamza@email.com, Role: Student	Account created; redirect to login.	Pass
2	Register with duplicate email.	Email: hamza@email.com (again)	'Email already registered' error shown.	Pass
3	Login with valid credentials.	Email + Password (correct)	JWT issued; user redirected to dashboard.	Pass
4	Start Pomodoro timer in Focus Room.	Click 'Start Timer'	25-minute countdown begins.	Pass
5	Find a study buddy by subject.	Topic: 'Database Systems'	Matched peers list displayed.	Pass
6	Send a buddy session request.	Click 'Send Request' on matched peer	Request sent; peer receives notification.	Pass
7	View available mentors.	Navigate to Mentorship module	List of mentors with subjects and availability shown.	Pass
8	Book a mentorship session.	Select mentor + available time slot	Session request created with 'Pending' status.	Pass
9	Instructor accepts session request.	Click 'Accept' on pending request	Status updated to 'Confirmed'; student notified.	Pass
10	Generate AI notes for a topic.	Prompt: 'Explain MongoDB indexing'	Notes generated and displayed in chat.	Pass

No	Test Case	Input / Action	Expected Result	Result
11	Create a community post.	Title + Body + Tags entered	Post published in forum feed.	Pass
12	Upload a resource to Resource Hub.	PDF file + metadata uploaded	File listed in Resource Hub.	Pass
13	Download a resource.	Click 'Download' on listed resource	File transfer initiated on device.	Pass
14	Join an existing study room.	Select room from list and join	Room interface loads; added to participants.	Pass
15	Logout from the platform.	Click 'Logout'	JWT invalidated; redirected to login.	Pass

6.1.4. Integration Testing

Integration testing verifies the communication that takes place between the various modules of the StudyBuddy system, assuring data flow between components and a correct module data boundaries are correctly handled.

Table 6.6: Integration Test Cases

No.	Test Case	Input / Action	Expected Result	Result
1	Registration → Login integration.	Register then immediately login with same credentials.	Login succeeds; user reaches dashboard.	Pass
2	Login → Dashboard content loading.	Login with valid credentials.	Dashboard loads with personalized progress and stats.	Pass
3	Peer match → Study session creation.	Match found; send and accept request.	WebRTC session initialized; both users connected.	Pass
4	Mentorship booking → Instructor notification.	Student books session.	Instructor receives	Pass

No.	Test Case	Input / Action	Expected Result	Result
			notification of pending request.	
5	Session accept → Student notification.	Instructor accepts session.	Student receives confirmation notification.	Pass
6	AI content generation → Notes saved to DB.	Generate notes for 'React Hooks'.	Notes generated and stored in Notes DB.	Pass
7	Community post → Forum feed display.	Create a new post.	Post appears immediately in community feed.	Pass
8	Resource upload → Hub listing.	Upload PDF with metadata.	Resource appears in Resource Hub immediately.	Pass
9	Focus Room session → Dashboard stats update.	Complete a Pomodoro session.	Study hours and streak updated on dashboard.	Pass
10	Logout → Session invalidation.	Logout then attempt to access dashboard URL.	Redirected to login; protected route blocked.	Pass
11	Multiple concurrent users — system stability.	Simulate 50 concurrent user sessions.	System remains responsive without crashes.	Pass
12	Data transmission security.	Monitor API calls during login.	All data transmitted over HTTPS; no plain-text credentials.	Pass

6.2. Automated Testing

For repetitive and regression prone scenarios, automated testing was used, and this allows the team to test important flows in an efficient manner in iterative development cycles.

Table 6.7: Tools Used for Automated Testing

Tool Name	Description	Applied On	Results
Jest	JavaScript testing framework for unit and integration tests.	Auth logic, API controllers, service functions.	All unit tests passed.
React Testing Library	Tests React UI components and user interactions.	Dashboard, login form, content generator UI.	UI components rendered and interacted correctly.
Supertest	HTTP assertion library for testing Express.js API endpoints.	Auth endpoints, session booking, peer matching APIs.	All API endpoints returned correct status codes and responses.
Postman (Collection Runner)	API testing tool used to run automated collection of API test cases.	End-to-end API flows for all nine modules.	All API collections executed successfully.

7. Conclusion and Future Work

The overall development of the StuddyBuddy project and comparison of the final system to the initial objectives set in the research introduction is covered in this final chapter. Outlines the main achievements in terms of technology, covering both the execution of implementation and lessons learnt during the process of development. Scalability, reliability and further enhancements for future extension of the system is also taken into consideration in the chapter. Viewing it from a broader perspective, it's intended to prove the efficacy of the suggested architecture, to shed light on future directions of the application development.

7.1. Conclusion

StudyBuddy does exactly what it promises – provides students with A unified, accessible and intelligent learning ecosystem that they are seeking to overcome the ‘fractured learning landscape', current EdTech solutions. By combining AI-powered content generation, real-time peer bringing together collaboration, structured mentorship and a community knowledge portal into one.

StudyBuddy is one end-to-end solution that eliminates the need to switch between a myriad of disconnected systems, including those used by students. Each of the 9 core modules on the platform was created, developed and tested extensively – ranging from Authentication & User Management to Student Dashboard, Content Generator, Focus Room, Study With Buddy, Mentorship System, Study Room through to Community & Forums, Resource Hub etc. Overall, the three-tier architecture (React front end, Express and Node.js backend, and MongoDB database) guarantees scalability, maintainability, and separation of concerns.

By embedding the locally-hosted Gemma3 AI model using Ollama, the project effectively put the power of AI right on their desk, incorporating tools that, in the future, could provide valuable assistance for constrained resources or isolated environments.By integrating with Ollama, the locally-hosted Gemma3 AI model, the project reduced reliance on expensive cloud-based APIs while saving on usage fees, making AI more accessible to users.

The Layer (WebRTC/Agora) that offers real-time communication puts into high-quality, low-latency video and audio peer sessions and mentorship sessions in front of mentors, improving the entire experience. The project is a great example from the software engineering point of view, spanning from requirement elicitation stage to system design, to implementation, till testing and documentation.

StudyBuddy is NOT just a Final Year Project, it is a possibility for a viable real-world

A product that is in education that has a good product-market fit and could be a viable business venture.

7.2. Future Work

The MVP is a good reference point at this time. The following improvements will be added in later versions of StudyBuddy:

7.2.1. Gamification

Talk through adding experience points (XP), achievement badges and leader boards to the game.Discuss awarding experience points (XP), achievement badges and leader boards to the game helps to promote positive study habits. Study streaks and tracking XP is already included in the dashboard. This infrastructure will be a stepping stone towards development, because on top of it a complete gamification layer will be added.

7.2.2. Advanced AI-Based Peer Matching

The available peer matching system is a rule-based one, which relies on subjects, availability. Future versions will feature machine learning which will compare peers according to learning style, past studies, quiz results and schedule compatibility significantly more personalized matches.

7.2.3. Mobile Application

In the future version, a dedicated iOS and Android mobile application will be developed, either as a React Native port of the existing React codebase or as a Flutter-based cross-platform solution. Students in areas where desktop support is not available will be able to greatly benefit from it.

7.2.4. Multilingual Support

The first edition will be available only in English. Urdu, Arabic and other languages will be added in future versions. To serve the needs of a wider audience of global students, especially in South Asia, in regional languages and the Middle East where StudyBuddy's target market is found.

7.2.5. AI-Assisted Mentorship Scheduling

Mentors are manually scheduled at this time. A future AI scheduling tool will consider mentor availability, student timezone, session history and subject demand to automatically recommend best session times for both players.

7.2.6. Analytics Dashboard for Instructors

A dedicated module for the instructor will be made available to provide insights into student Students to track their activities and progress through mentorship sessions, quiz results, and study room attendance, and data-driven teaching decisions.

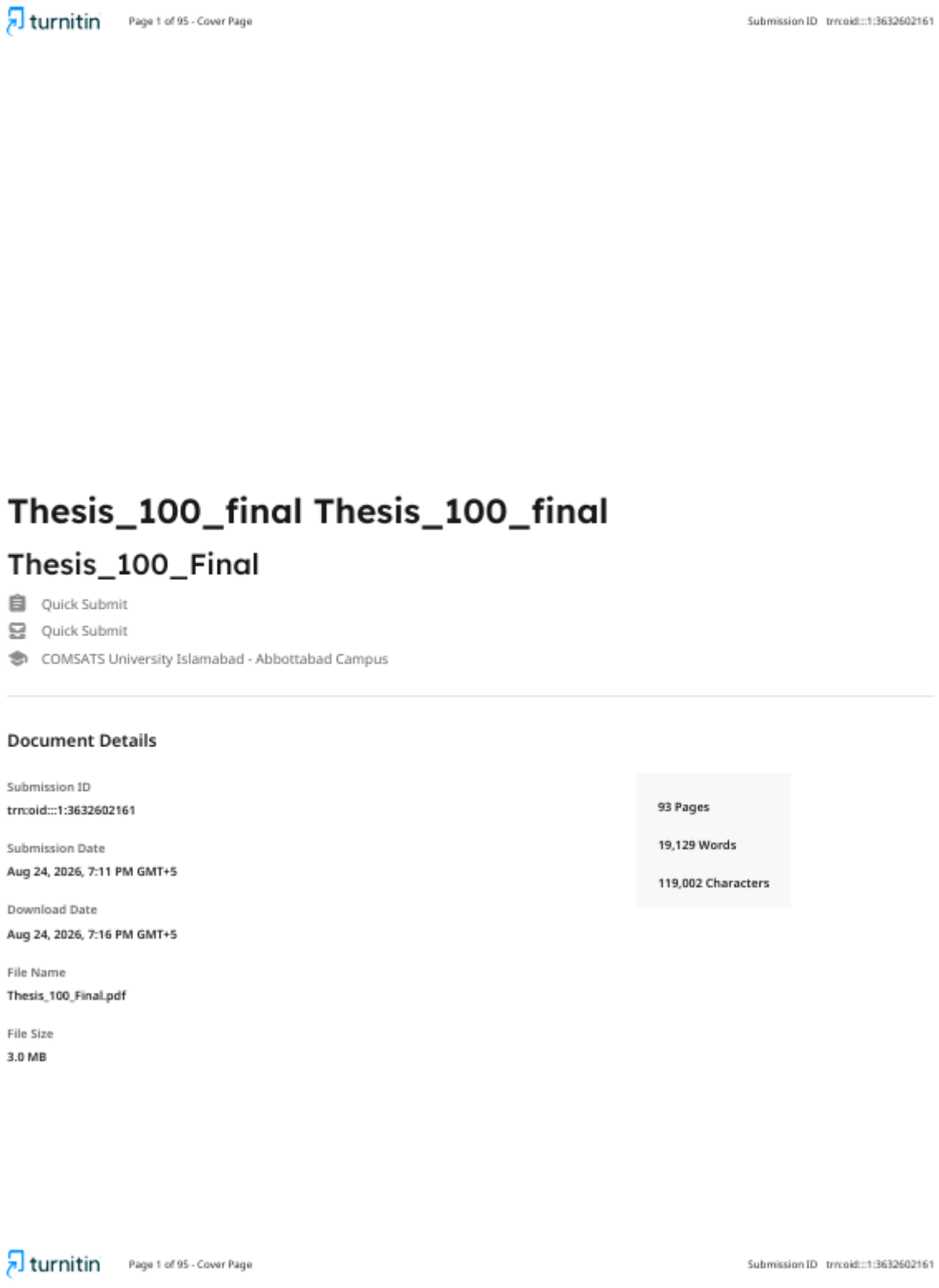
7.2.7. Advanced Content Generation

Future AI enhancements will include file upload-based note generation (PDFs, images of handwritten notes via OCR), flashcard creation, and adaptive quiz difficulty based on historical performance data.

References

This chapter presents the references and technical resources that supported the design and development of the StudyBuddy platform. These sources included guidance on various aspects of AI content generation, user authentication, real-time communication, cloud storage, and the development of a complete web application. In addition to their own documentation, the references include the official website documentation for core technologies used in the project like: Ollama, React, Node.js, Express.js, MongoDB, Socket.io, WebRTC, and Google Identity Services, along with standard software engineering textbooks, all presented in a consistent academic format.

- OpenAI, "Ollama Documentation," 2025. [Online]. Available: https://ollama.com/. [Accessed 4 July 2026].
- Google, "Google Identity Services Documentation," 2025. [Online]. Available: https://developers.google.com/identity. [Accessed 4 July 2026].
- MongoDB, "MongoDB Documentation," 2025. [Online]. Available: https://www.mongodb.com/docs/. [Accessed 4 July 2026].
- Node.js, "Node.js Documentation," 2025. [Online]. Available: https://nodejs.org/docs/latest/api/. [Accessed 4 July 2026].
- React, "React Documentation," 2025. [Online]. Available: https://react.dev/. [Accessed 4 July 2026].
- Socket.IO, "Socket.IO Documentation," 2025. [Online]. Available: https://socket.io/docs/. [Accessed 4 July 2026].
- World Wide Web Consortium, "WebRTC API Documentation," 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API. [Accessed 4 July 2026].
- Amazon Web Services, "Amazon S3 Documentation," 2025. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/. [Accessed 4 July 2026].
- R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach* (9th Edition), New York, USA: McGraw-Hill Education, 2020.
- MongoDB, "MongoDB University," 2025. [Online]. Available: https://learn.mongodb.com/. [Accessed 4 July 2026].
- bcrypt, "bcrypt Documentation," 2025. [Online]. Available: https://www.npmjs.com/package/bcrypt. [Accessed 4 July 2026].
- JWT, "JSON Web Tokens Introduction," 2025. [Online]. Available: https://jwt.io/introduction. [Accessed 4 July 2026].


turnitin Page 1 of 95 - Cover Page Submission ID trn:oid:::1:3632602161

Thesis_100_final Thesis_100_final

Thesis_100_Final

Quick Submit

Quick Submit

COMSATS University Islamabad - Abbottabad Campus

Document Details

Submission ID
trn:oid:::1:3632602161

Submission Date
Aug 24, 2026, 7:11 PM GMT+5

Download Date
Aug 24, 2026, 7:16 PM GMT+5

File Name
Thesis_100_Final.pdf

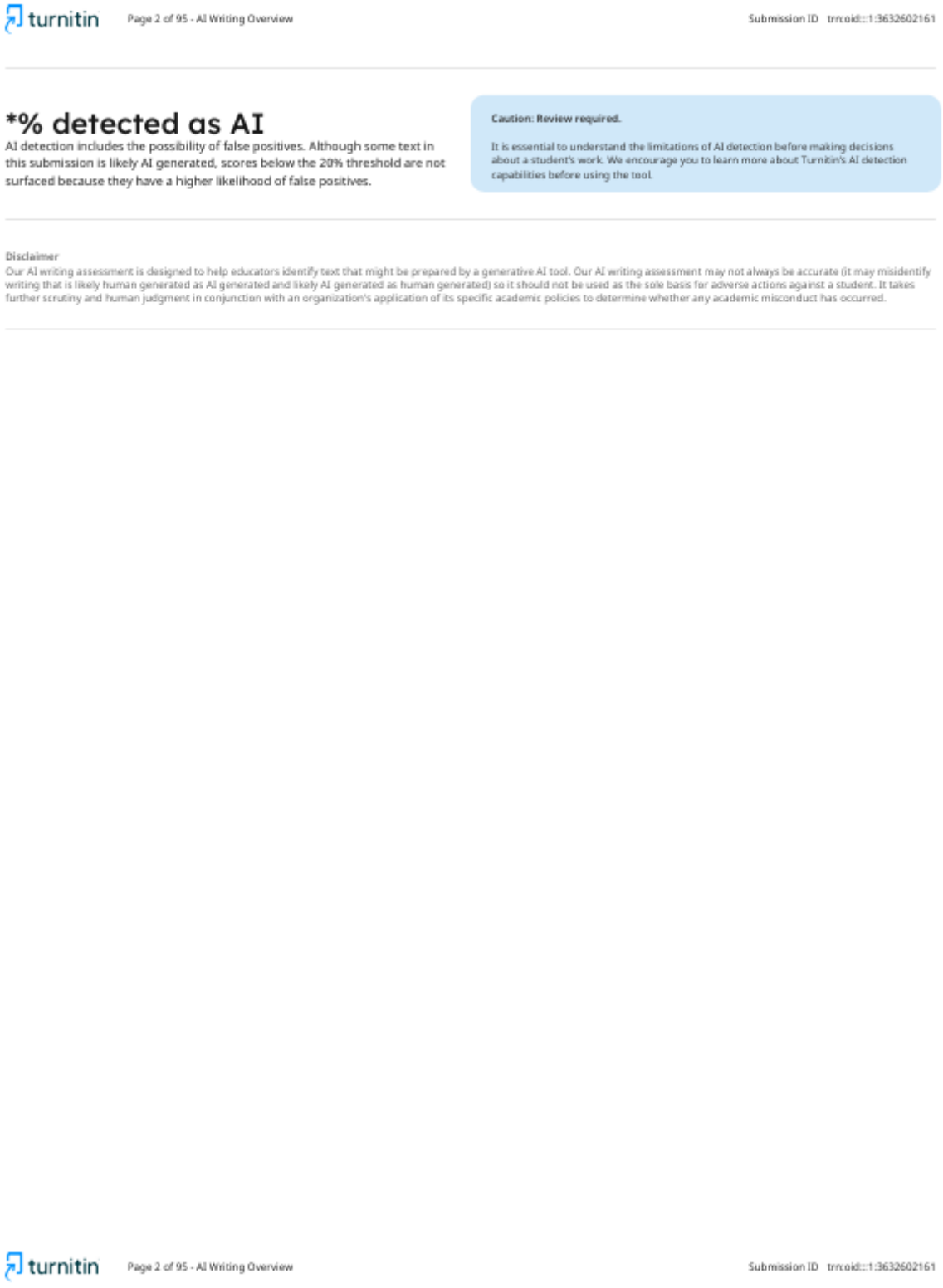
File Size
3.0 MB

93 Pages

19,129 Words

119,002 Characters

turnitin Page 1 of 95 - Cover Page Submission ID trn:oid:::1:3632602161



turnitin Page 2 of 95 - AI Writing Overview Submission ID trn:oid:::1:3632602161

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

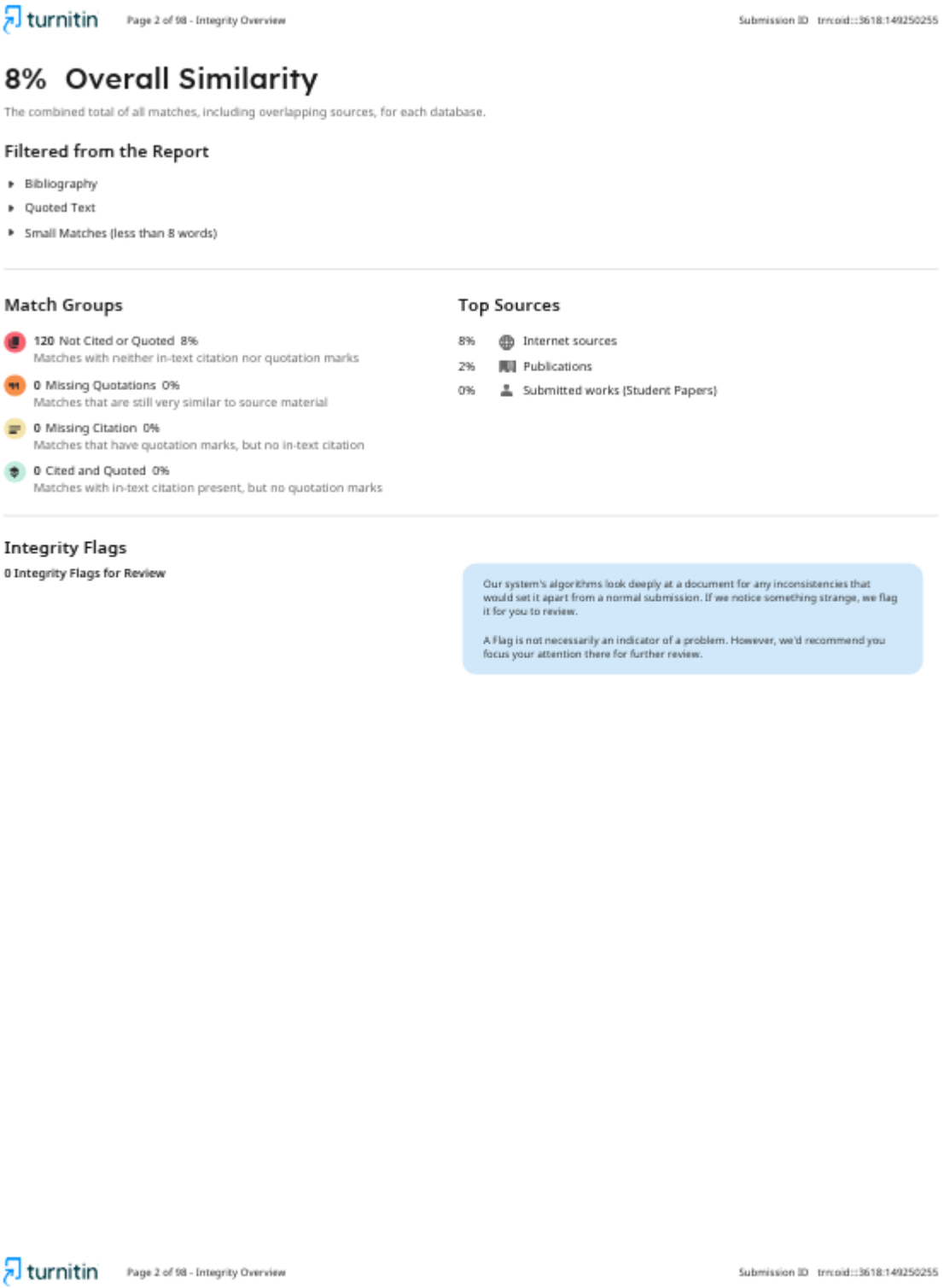
Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

turnitin Page 2 of 95 - AI Writing Overview Submission ID trn:oid:::1:3632602161



turnitin Page 2 of 98 - Integrity Overview Submission ID trn:oid:::3618:149250255

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

120 Not Cited or Quoted 8%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
2% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

turnitin Page 2 of 98 - Integrity Overview Submission ID trn:oid:::3618:149250255